

ENGINEERING HANDBOOK

AlloCat

Architecture, data flow and file reference

This handbook explains how the AlloCat personal-finance app works end to end: the offline-first data layer, the sync engine, the server and database, the Android SMS pipeline, notifications, AI, the admin portal and the UI system. It ends with a short description of every source file.

Repository	<code>allocat-web</code> (package name <code>AlloCat-web</code>)
Stack	Next.js 16 · React 19 · Supabase · Dexie · TanStack Query 5 · Tailwind v4 · Capacitor 8 · Serwist
Size	~400 source files · ~55k lines · 21 SQL migrations · IDB schema v19
Generated	29 September 2026, from branch <code>main</code> (commit 6e31f79)

Contents

1. What AlloCat is
2. The big picture
3. Repository layout
4. Routing, layouts and providers
5. Authentication and middleware
6. Offline-first data layer: the core idea
7. IndexedDB schema (Dexie)
8. The read path
9. The write path and mutation hooks
10. SyncEngine: draining the queue
11. Hydration: pulling server changes
12. SyncProvider: lifecycle and refresh
13. Worked example: a budget item created offline
14. Server actions and cascades
15. Database (Supabase / Postgres)
16. Android native shell
17. SMS transaction tracking
18. Notifications and push
19. AI features
20. Budget domain: templates, carry-forward, quiz
21. Admin portal
22. Runtime flags, force update, support
23. PWA and service worker
24. UI system, theme and tour
25. Testing, tooling and environment
26. Rules of thumb for contributors
27. File-by-file reference
28. Glossary

1 What AlloCat is

AlloCat is a minimalist personal-finance app. Users set a monthly budget, split it into categories and items, log spending, track savings goals, assets, net worth and debts, and ask an AI assistant about their money. It ships in two forms from one codebase:

- **Web / PWA:** a Next.js app that works in the browser and can be installed as a progressive web app.
- **Android app:** a Capacitor shell that loads the same deployed web app in a WebView and adds one native feature: it reads incoming bank and UPI transaction SMS and logs spends automatically.

The app is free for everyone. There are no tiers or paywalls. The only money surface is an optional Ko-fi tip, which unlocks nothing except a cosmetic crown badge.

Technology stack

Layer	Technology	Role in AlloCat
Framework	Next.js 16 (App Router), React 19	Pages, server actions, API routes, SSR
Backend	Supabase (Postgres, Auth, RLS)	Source of truth, login, row-level security
Local database	Dexie 4 over IndexedDB	Offline copy of all user data plus the sync queue
Client cache	TanStack Query 5	In-memory cache on top of IndexedDB
Styling	Tailwind v4, next-themes, motion, vaul	Neo · Lime design system, light/dark, animations, bottom sheets
Native	Capacitor 8 (Android), Java plugin	WebView shell, SMS receiver, local notifications, haptics, FCM
PWA	Servist	Service worker, precache, offline page
AI	OpenRouter	Chat, weekly insight, monthly summary, overspend copy
Push	web-push (VAPID), FCM HTTP v1	Browser push and Android push
Hosting	Vercel (+ cron)	Deploys the app; nightly Play install sync
Tests	Vitest	Unit tests next to source (<code>*.test.ts</code>)

2 The big picture

Everything in AlloCat is built around one principle: **the screen always reads from the device**. IndexedDB holds a full copy of the user's data. The network is used for two background jobs: pushing local changes up (the sync engine) and pulling server changes down (hydration).

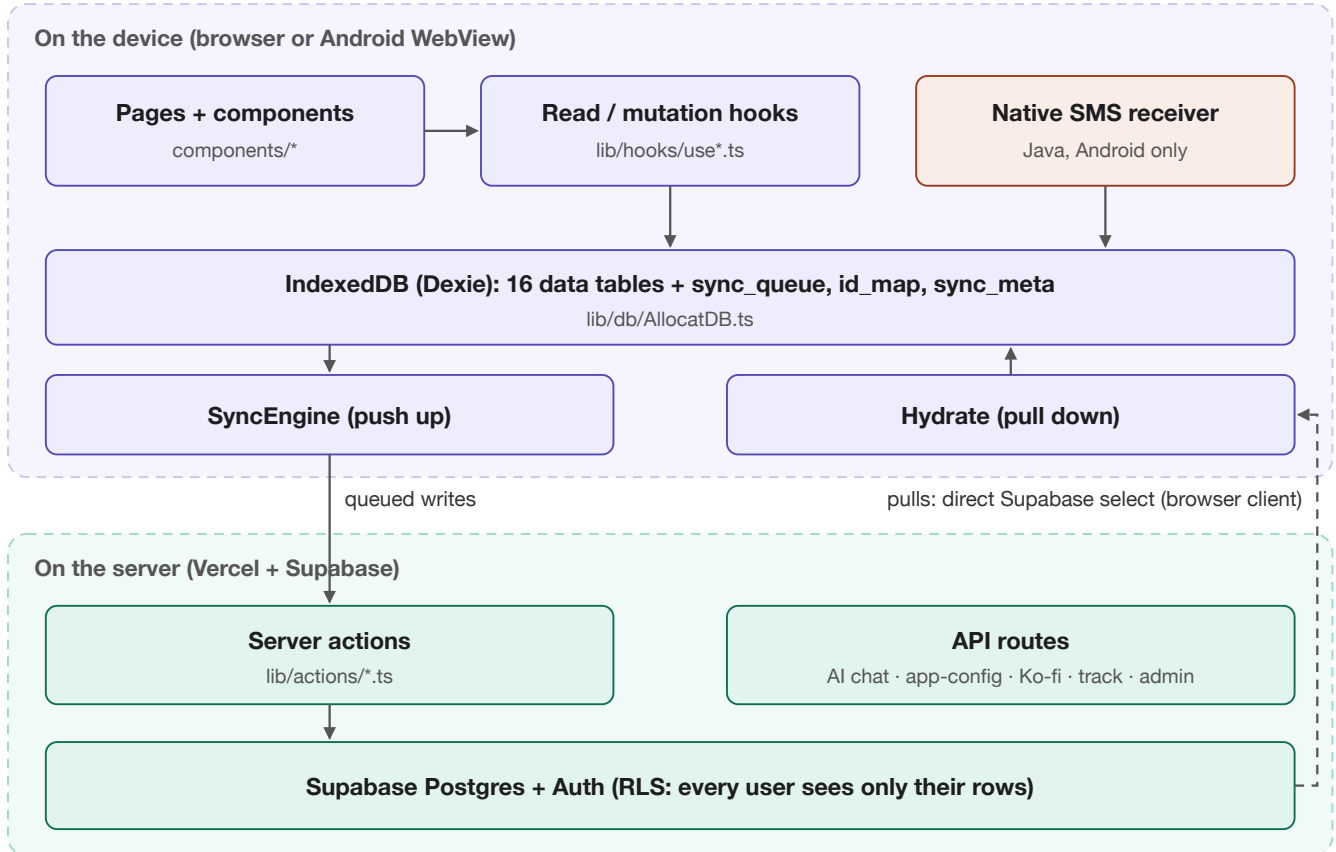


Figure 1. System overview. Purple: device. Coral: native Android. Teal: server.

Hydration reads Supabase directly with the browser client (RLS restricts it to the signed-in user's rows). All writes go through server actions, called by the SyncEngine.

The five sentences to remember

1. Every page reads from IndexedDB first; the server is only a fallback on a cache miss.
2. Every write lands in IndexedDB immediately (optimistic), then goes into `sync_queue`.
3. `SyncEngine` drains the queue by calling server actions, swaps temporary ids for real ones, retries, and rolls back if a write fails for good.
4. `hydrateAllTables()` pulls only rows changed since the last pull (delta by `updated_at`), and an id-only sweep catches deletions.
5. On Android, a native receiver catches transaction SMS even when the app is killed; the web layer parses and logs them on-device, and only extracted fields leave the phone.

3 Repository layout

allocat-web/	
├─ app/	Next.js App Router
│ └─ (app)/	protected app shell: dashboard, budget, debt, goals, net-worth, activity, notifications, profile, reports, sms, support, transactions
│ └─ admin/	internal admin portal (RSC + server actions only)
│ └─ api/	route handlers: ai/chat, app-config, kofi/webhook, track, admin/*
│ └─ auth/	login, signup, OAuth callback
│ └─ onboarding/	post-signup deck + budget quiz
│ └─ legal/	privacy policy, delete-account
│ └─ share-target/	PWA Web Share Target endpoint
│ └─ layout.tsx	root layout (fonts, theme, providers)
│ └─ manifest.ts	PWA manifest
│ └─ sw.ts	service worker source (Serwist)
├─ components/	UI, grouped by feature (budget/, debt/, sms/, pwa/, ui/, ...)
├─ lib/	
│ └─ db/	Dexie schema, hydrate, prefetch ← offline-first core
│ └─ sync/	SyncEngine, reconcile, device fields ← offline-first core
│ └─ providers/	SyncProvider, QueryProvider, theme/accent/currency
│ └─ hooks/	read + mutation hooks per domain
│ └─ actions/	server actions ("use server")
│ └─ supabase/	server / browser / service clients, middleware helper
│ └─ sms/	SMS ingest, matching, rule resolution, native mirror
│ └─ ai/	SMS parser, chat guard, prompt builders
│ └─ budget/	carry-forward, templates, setup math, quiz mapping
│ └─ server/	push, FCM, OpenRouter, activity logger, rate limit
│ └─ admin/	admin guard, queries, actions
│ └─ native/	Capacitor plugin bindings, local notifications
│ └─ notify/	local notification inbox + message pools
│ └─ config/ theme/ tour/ play/ currency/ utils/ types/ ...	
├─ android/	Capacitor Android project (Java SMS plugin)
├─ supabase/migrations/	SQL migrations
├─ docs/	design docs, specs, Play compliance
├─ public/	icons, fonts, sounds, Lottie, avatars, sw.js
└─ proxy.ts	Next 16 middleware (named proxy)

4 Routing, layouts and providers

AlloCat uses the Next.js App Router. Route groups separate the signed-in shell from auth, onboarding and admin.

Path	What it is	Auth
<code>/dashboard</code> , <code>/budget</code> , <code>/budget/[categoryId]</code> , <code>/debt</code> , <code>/net-worth</code>	Core app screens	Redirect to login if signed out (middleware)
<code>/goals</code> , <code>/profile</code> , <code>/activity</code> , <code>/sms</code> , <code>/reports</code> , <code>/transactions</code> , <code>/notifications</code> , <code>/support</code>	Other app screens	Not in the middleware list; data is still per-user via RLS
<code>/auth/login</code> , <code>/auth/signup</code> , <code>/auth/callback</code>	Sign-in flows	Signed-in users are redirected to <code>/dashboard</code>
<code>/onboarding</code>	Slides + budget quiz	Protected
<code>/admin/*</code>	Internal portal	Protected + <code>requireAdmin()</code> (404 for non-admins)
<code>/api/*</code>	Route handlers	Each handler authenticates itself
<code>/share-target</code> , <code>/legal/*</code> , <code>/~offline</code>	PWA share, legal pages, offline fallback	Public

Provider tree

Providers are split between the root layout (things every page needs, including auth pages) and the app layout (things only signed-in screens need, most importantly the sync layer).

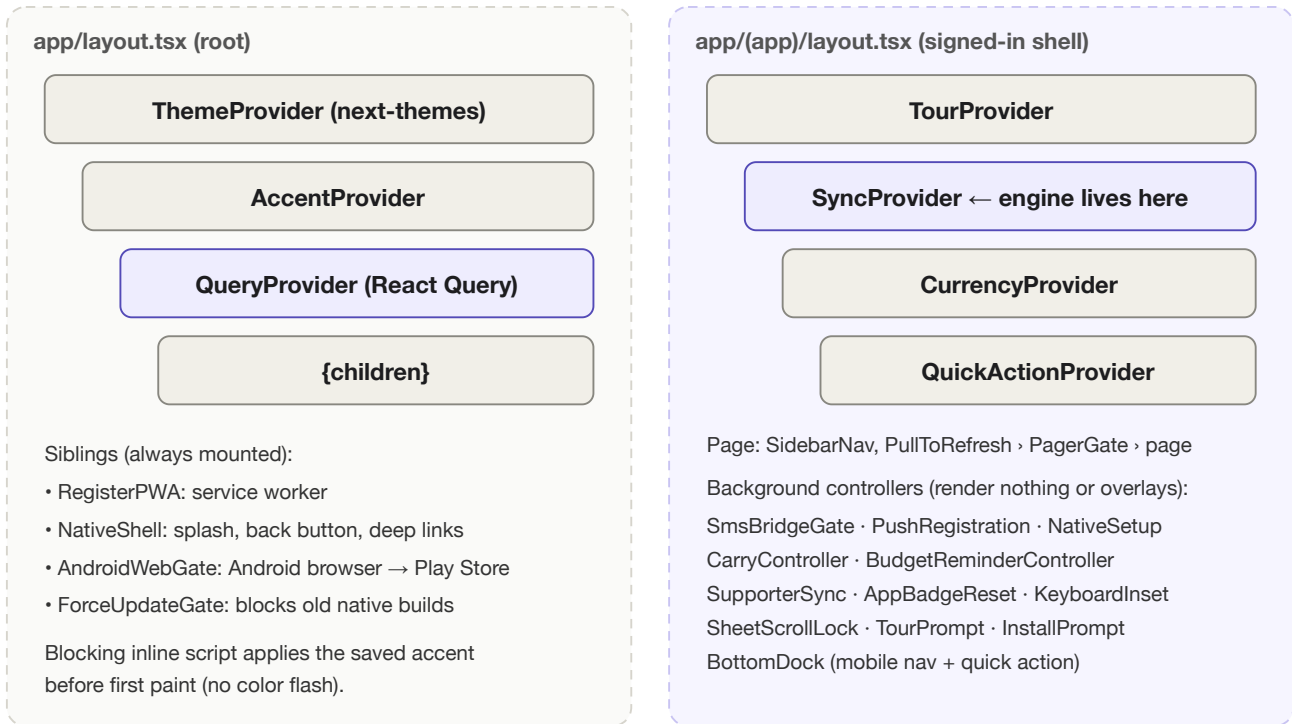


Figure 2. Provider nesting. Only the app shell mounts SyncProvider, so /auth, /onboarding and /legal never start the sync engine.

Layout behaviour

- The shell is a 480px-wide mobile frame, and becomes a sidebar layout at the `md:` breakpoint on desktop.
- On mobile the document itself scrolls (not an inner container), so the glass bottom dock's blur can sample the content behind it.
- On native Android, at one of the 5 tab roots, `PagerGate` renders `TabPage`: all five tabs live side by side in a swipe pager. The tab order comes from `lib/nav/navItems.ts`.

5 Authentication and middleware

Auth uses Supabase with cookie-based sessions through `@supabase/ssr`. There are three clients:

Client	File	Used by
Server client	<code>lib/supabase/server.ts</code>	Server actions, RSCs, route handlers. Reads and writes the session cookie.
Browser client	<code>lib/supabase/client.ts</code>	Hydration pulls and <code>auth.getSession()</code> on the device.
Service client	<code>lib/supabase/service.ts</code>	Server-only, bypasses RLS: admin, push fan-out, AI quota, Ko-fi.

The middleware is named `proxy.ts`

In Next 16 the middleware file lives at the repo root as `proxy.ts` and exports `proxy()`. It calls `updateSession()`, which refreshes the auth token and redirects. Every request it runs on costs a network call to Supabase Auth, so it is kept narrow.

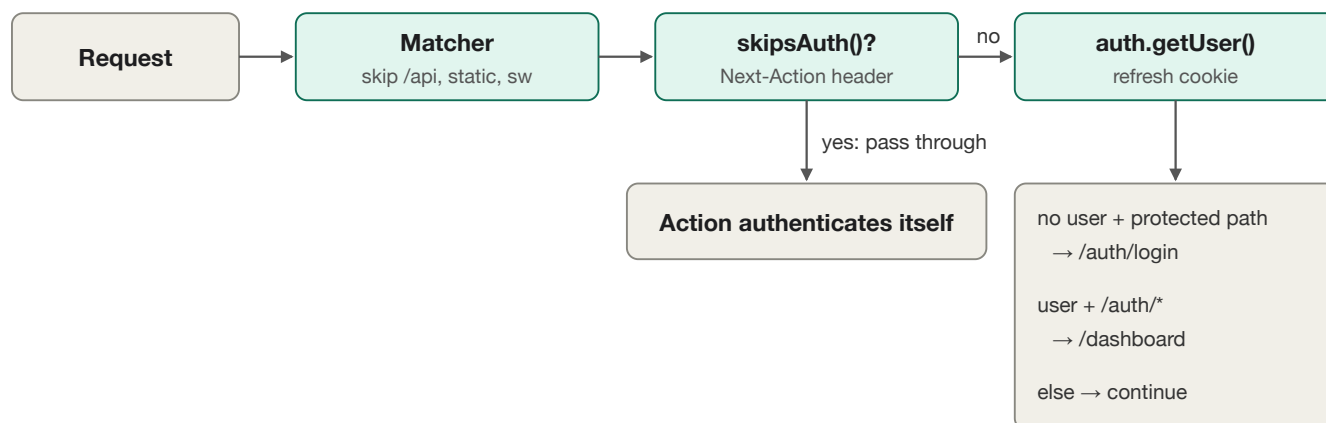


Figure 3. Middleware decision flow for a document or RSC request.

```

// lib/supabase/middleware.ts
export function skipsAuth(request: NextRequest): boolean {
  // Server action POST – carries the Next-Action header.
  if (request.headers.get("next-action")) return true;
  // Route handlers authenticate themselves (or use the service role).
  if (request.nextUrl.pathname.startsWith("/api/")) return true;
  return false;
}

```

Why skip server actions? A server action can refresh the cookie itself, and it verifies the user anyway. Before this change, a 40-item sync drain made 40 extra auth calls.

Request-scoped auth on the server

Both `createClient()` and `getAuthedUser()` are wrapped in React `cache()`. One request verifies the user once, even when actions call other actions. An SMS ingest reaches `quickLogSpend`, then `addAssetEntry` or `makePayment`, then a notification. `cache()` never spans requests, so two users can never share a result.

```
// lib/supabase/server.ts
export const getAuthedUser = cache(async () => {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  return user;
});
```

Rule: use `getAuthedUser()` in new server actions. A bare `supabase.auth.getUser()` adds a network hop per call. On the device, single-table refreshes read the user id from `auth.getSession()` (local), never `getUser()`.

Logout and account switching

`lib/auth/clearSession.ts` runs on logout. It flushes the sync queue first (so a just-made change reaches the server), then calls `clearDB()` and removes account-scoped `localStorage` keys. Device-level keys survive: the device id used for SMS dedupe, the theme and the accent. If a different user signs in on the same device, hydration notices the `__userId__` mismatch and wipes IndexedDB first.

6 Offline-first data layer: the core idea

This is the most important chapter. Read it before touching any data flow.

Three layers cooperate:

1. **IDB cache.** Dexie tables mirror the Supabase tables, plus three sync tables.
2. **Hydration and prefetch.** Server rows are pulled into IDB, then the React Query cache is warmed from IDB.
3. **Mutation queue.** Local writes are queued and replayed to the server by the SyncEngine.

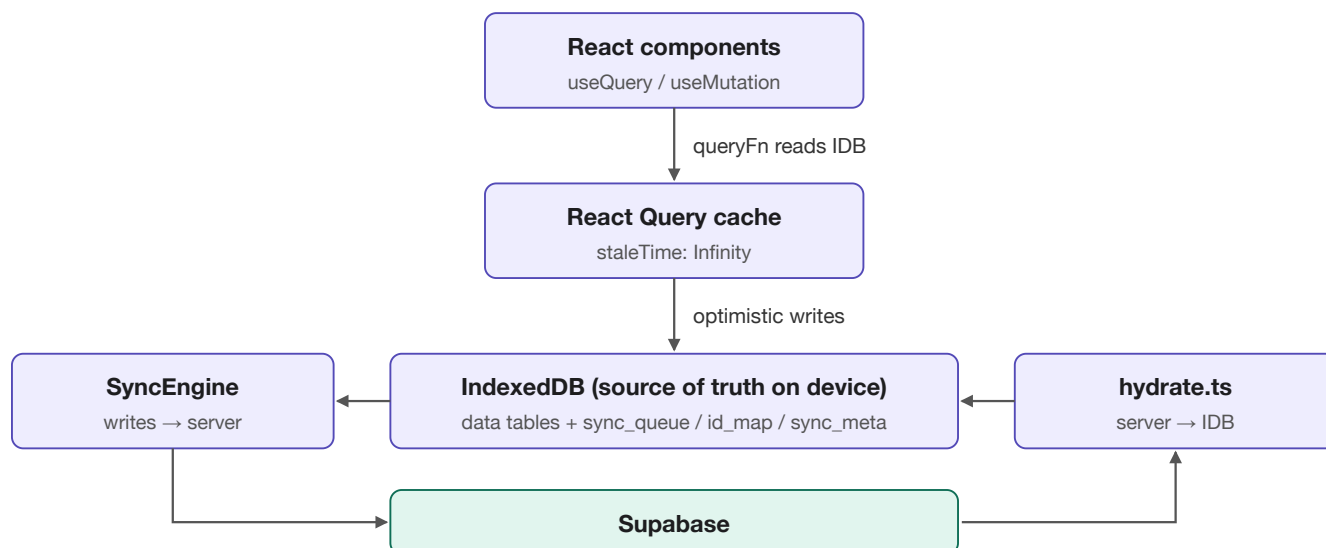


Figure 4. The three layers. UI never waits for the network.

Why this design?

- **Instant screens.** The first paint comes from local data, with no loading spinners after the first launch.
- **Works offline.** A phone on a bad connection can still log spends; they sync later.
- **Native-ready.** The Android app is a WebView of the same site, so the offline layer carries over unchanged.
- **Cheap reconciles.** Delta pulls move almost no data when nothing changed.

7 IndexedDB schema (Dexie)

`lib/db/AllocatDB.ts` defines the database. It is currently at **schema version 19**. `getDB()` (`lib/db/index.ts`) returns a browser-only singleton and throws on the server.

Data tables (mirror Supabase)

Table	Holds	Hydrate behaviour
profiles	Name, currency, avatar, onboarding, supporter flag	Full + delete sweep
budgets	One row per (user, month, year): total, template id	Full + delete sweep
categories	Budget categories (needs, wants, investments, misc), allocation, icon, color	Full + delete sweep
budget_items	Items: planned vs actual amount, link to an asset or debt, template identity, overspend count	Full + delete sweep
assets / asset_categories	Assets and goals (<code>is_goal</code>), grouping	Full + delete sweep
asset_value_history	Asset ledger entries with running total	First pull capped at 500, no sweep
debts	Owed (internal/external) and lent money, interest, EMI	Full + delete sweep
reports	Monthly report notes and summary	Full + delete sweep
net_worth_snapshots	Daily net worth snapshots	First pull capped at 24, no sweep
activity_logs	History of mutations for the activity feed	First pull capped at 200, no sweep
merchant_rules	Learned merchant → budget item rules	Full + delete sweep
sms_transactions	All transactions (SMS and manual)	First pull capped at 200, no sweep
sms_blocklist	Hashed SMS template keys reported as "not a transaction"	Full + delete sweep
feedback	User feedback rows	First pull capped at 100, no sweep
notifications	Local only: the device notification inbox	Never synced

Sync infrastructure tables

```
// lib/db/AllocatDB.ts
export interface SyncQueueItem {
  id?: number; // auto-increment
  table: SyncTable;
  operation: SyncOperation; // INSERT | UPDATE | DELETE | PAYMENT | BULK_SETUP |
  // CARRY_SETUP | UNDO_CARRY | ACHIEVE | CATEGORIZE | ...
  recordId: string; // local IDB id (temp_id for INSERTs)
  tempId?: string; // only for INSERTs
  payload: Record<string, unknown>; // arguments for the server action
  retries: number;
  status: "pending" | "processing" | "done" | "failed";
  createdAt: number;
  lastError?: string;
  nextAttemptAt?: number; // backoff deadline
}
export interface IdMapEntry { tempId: string; realId: string; table: SyncTable; }
export interface SyncMetaEntry {
  table: string; // or "__userId__", or "__carry__<y>--<m>"
  lastSynced: number;
  watermark?: string; // newest server updated_at pulled (delta sync)
  userId?: string; // only on __userId__ (account-switch detection)
  carry?: { state: "carried" | "undone" | "dismissed"; sourceLabel: string; budgetId: string };
}
```

Table	Purpose
<code>sync_queue</code>	Pending writes, in order. Drained by SyncEngine.
<code>id_map</code>	<code>temp_<uuid></code> → real server id, written when an INSERT syncs.
<code>sync_meta</code>	Per-table <code>lastSynced</code> and delta <code>watermark</code> , the current user id, and carry-forward markers.

Schema changes: add a new `this.version(N).stores({...})` block in the constructor. Never edit an earlier version block; existing devices upgrade by replaying versions in order. Version 19, for example, only deletes the `profiles` watermark so profiles do one full re-pull.

8 The read path

Read hooks live in `lib/hooks/use<Domain>.ts`. Each one follows the same pattern:

1. The hook calls `useQuery` with an exported key (e.g. `budgetKey(month, year)`, `DASHBOARD_KEY`).
2. The `queryFn` calls `getXFromIDB()`. If IDB has the data, return it immediately.
3. On a miss (first launch, not hydrated yet), fall back to a read-only server action.

```
// lib/hooks/useBudget.ts
export function useBudgetData(month: number, year: number) {
  return useQuery({
    queryKey: budgetKey(month, year),
    queryFn: async () => {
      // Serve from IDB first – instant, works offline
      const local = await getBudgetFromIDB(month, year);
      if (local) return local;
      // IDB miss: read-only server fetch (never creates a row).
      const view = await getBudgetView(month, year);
      if (view) return view;
      return { id: "", month, year, totalBudget: 0, templateId: null, categories: [] };
    },
  });
}
```

`getBudgetFromIDB` reads the budget by the compound `[month+year]` index, then all its categories, then **all items in one bulk query** (`anyOf(categoryIds)`), grouped in memory. This avoids N+1 queries.

React Query settings

```
// lib/providers/QueryProvider.tsx
new QueryClient({ defaultOptions: { queries: {
  staleTime: Infinity,           // IDB is the source of truth
  gcTime: 10 * 60 * 1000,
  refetchOnWindowFocus: false, // IDB invalidation handles freshness
  retry: 1,
}}});
```

Because nothing is ever "stale" by timer, freshness comes from explicit `invalidateQueries` calls after writes, syncs and hydrations.

Prefetch

After hydration, `prefetchAllQueries()` (`lib/db/prefetch.ts`) warms the dashboard, current-month budget, net worth and debt queries from IDB, so the first navigation shows no skeletons.

Rule: use `qc.fetchQuery` , not `prefetchQuery` , when adding prefetched keys. With `staleTime: Infinity` , `prefetchQuery` would skip keys that are already cached and serve stale data across reloads.

9 The write path and mutation hooks

Every mutation hook does three things, in this order:

1. **Write to IDB optimistically.** New rows get a `temp_<uuid>` id.
2. **Enqueue** a `SyncQueueItem` through `useEnqueue()`.
3. **Invalidate** the matching query keys in `onSuccess`, so the UI re-reads IDB.

```
// lib/hooks/useBudget.ts - useAddBudgetItem (trimmed)
mutationFn: async ({ categoryId, name, planned = 0, link = null, emoji = null }) => {
  const db = getDB();
  const tempId = `temp_${crypto.randomUUID()}`;
  const now = new Date().toISOString();

  // 1. optimistic IDB row
  await db.budget_items.add({
    id: tempId, category_id: categoryId, user_id: "__pending__",
    name: name.trim(), planned_amount: planned, actual_amount: 0, /* ... */
    created_at: now, updated_at: now,
  });

  // 2. queue the server write
  await enqueue({
    table: "budget_items", operation: "INSERT",
    recordId: tempId, tempId,
    payload: { categoryId, name: name.trim(), planned, link, emoji },
  });
  return { id: tempId };
},

// 3. refresh screens that read this data
onSuccess: (_d, { month, year }) => {
  qc.invalidateQueries({ queryKey: budgetKey(month, year) });
  qc.invalidateQueries({ queryKey: ["categoryData"] });
  qc.invalidateQueries({ queryKey: ["categoryItems"] });
},
```

An UPDATE looks the same without the temp id: `db.categories.update(id, {icon})`, then enqueue `{table: "categories", operation: "UPDATE", recordId: id, payload: {categoryId, updates: {icon}}}`.

If a UI flow needs a cascade that the server also performs (e.g. spending on an item linked to an asset or debt), `lib/utils/budget-cascade.ts` mirrors that cascade in IDB so the UI updates instantly. The server stays authoritative.

10 SyncEngine: draining the queue

`lib/sync/SyncEngine.ts` (about 1,200 lines) is the write-side heart of the app. One instance is created by `SyncProvider`. It starts draining on `start()`, on every `enqueue()` when online, on the browser `online` event, and when a backoff timer fires.

The dispatch table

Each `(table, operation)` pair maps to a server action. If a pair is missing, the item fails permanently.

```
private dispatch: Dispatcher = {
  budgets: {
    INSERT:      (p) => ensureBudgetRow(p.month, p.year),
    UPDATE:      (p) => updateBudgetTotal(p.budgetId, p.totalAmount),
    BULK_SETUP:  (p) => setupBudgetFromTemplate(p.budgetId, p.totalBudget, p.categories,
p.templateId),
    CARRY_SETUP: (p) => carryBudgetForward(p),
    UNDO_CARRY:  (p) => undoCarriedBudget(p.budgetId),
    STAMP_TEMPLATE: (p) => stampBudgetTemplateIdentity(p),
  },
  categories: { INSERT: ...addBudgetCategory, UPDATE: ...(icon|color|name|allocated_amount), DELETE: ...
deleteCategory },
  budget_items: { INSERT: ..., UPDATE: ..., DELETE: ..., PAYMENT: ...quickLogSpend },
  assets / asset_categories / debts / sms_transactions / ... : { ... },
};

// Batched pairs: ONE server call for many queue items
private bulkDispatch = {
  "sms_transactions:INSERT": (payloads) => ingestSmsTransactionsBulk(payloads),
};
```

Adding a new table or operation? Register it in `dispatch`, or every queued item for it will fail after 3 retries and roll back.

The drain loop

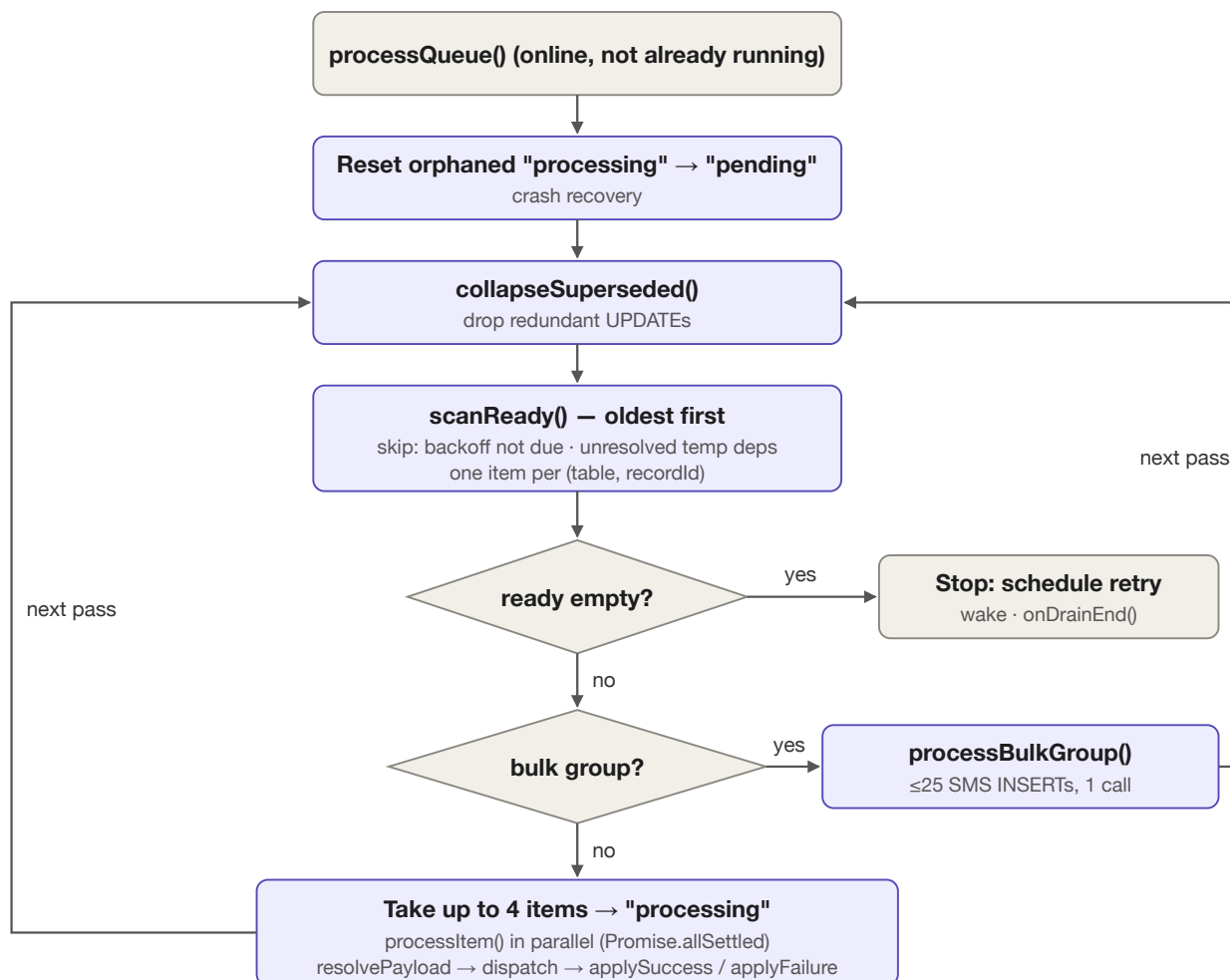


Figure 5. One drain. Passes repeat until no item is ready.

Ordering guarantees

- **Producer before dependent.** `extractTempIds()` walks a payload for strings starting with `temp_`. If any has no `id_map` entry yet, the item waits for a later pass. If the producing INSERT has failed and is gone, the dependent is "doomed": it is failed and rolled back so it does not block the queue.
- **Same record stays serial.** Only the oldest pending item per `(table, recordId)` is admitted per pass, so two edits to the same row never overlap or reorder.
- **Bounded concurrency.** Independent items run up to `MAX_CONCURRENCY = 4` at a time.

Temp ids and the id map

```
// Before every dispatch: rewrite temp ids anywhere in the payload
private async resolvePayload(payload) {
  const resolve = async (obj) => {
    if (typeof obj === "string" && obj.startsWith("temp_")) {
      const mapping = await db.id_map.get(obj);
      return mapping ? mapping.realId : obj;
    }
    // ...recurse into objects and arrays...
  };
  return resolve(payload);
}
```

When an INSERT succeeds, `applySuccess()` :

1. writes `id_map[tempId] = realId`;
2. calls `replaceIDBRecord()` : reads the local row, deletes it, and puts the merged row under the real id.
`reconcileInsertReplacement()` keeps local state the user changed after the insert (e.g. an SMS allocated before its INSERT synced);
3. calls `rewriteChildForeignKeys()` : children already in IDB that point at the temp id (categories → budget, items → category, assets → asset category, items → linked asset or debt) are repointed to the real id.

`BULK_SETUP` and `CARRY_SETUP` create many rows at once; their results carry `categoryIdMap` and `itemIdMap` arrays and are applied the same way.

Retries, backoff and rollback

```
private async applyFailure(item, errMsg) {
  const nextRetries = item.retries + 1;
  if (nextRetries >= MAX_RETRIES) { // MAX_RETRIES = 3
    await db.sync_queue.update(item.id, { status: "failed", lastError: errMsg });
    await this.rollback(item); // undo the optimistic write
    this.callbacks.onRollback?.(item, errMsg); // SyncProvider invalidates queries
    return;
  }
  // no blocking sleep: set a deadline, later passes skip it until due
  await db.sync_queue.update(item.id, {
    status: "pending", retries: nextRetries, lastError: errMsg,
    nextAttemptAt: Date.now() + Math.pow(2, nextRetries) * 1000, // 2s, 4s
  });
}
```

Rollback deletes an optimistic INSERT row. For bulk setups, it deletes every temp category and item in the payload (and, for a carry, the temp budget and carry marker).

Request-shaping rules

Rule	What it does	Guard rails
Batched ops	<code>bulkDispatch</code> sends up to <code>MAX_BATCH</code> (25) queued SMS INSERTs in one call. The action returns one outcome per input, in order, so one bad item retries alone.	<code>onSynced</code> fires once per group. To batch a new op, add a bulk action and a <code>bulkDispatch</code> entry; never change per-item semantics.
Superseded writes dropped	<code>selectSupersededIds()</code> deletes an older UPDATE when the <i>immediately following</i> queued op on the same record writes the same field set. Five slider drags become one request.	Only tables in <code>COLLAPSIBLE_UPDATE_TABLES</code> (budgets, categories, budget_items, assets, debts, asset_categories). Never PAYMENT, CATEGORIZE, ACHIEVE or bulk ops. Never <code>actual_amount</code> (it cascades and logs activity).
One refresh per drain	The engine emits <code>onDrainEnd</code> when the queue comes to rest. SyncProvider collects tables to re-pull during the drain and flushes them once.	Isolated mutations (no drain running) use a 150 ms coalescing timer instead.

Callbacks to SyncProvider

Callback	When	Used for
<code>onPendingChange(count)</code>	Queue size changes	Pending-sync indicator
<code>onSynced(item)</code>	An item (or group) succeeded	Invalidate queries; schedule re-pulls of tables the server cascaded into
<code>onRollback(item, error)</code>	Permanent failure	Invalidate queries so the UI drops the optimistic state
<code>onDrainEnd()</code>	Queue at rest	Flush the coalesced forced refresh once

`flush()` is a best-effort drain used before logout, so an allocation made just before signing out reaches the server before IDB is wiped.

11 Hydration: pulling server changes

`lib/db/hydrate.ts` reconciles every table for the current user into IDB. It runs at startup, on foreground and reconnect, on pull-to-refresh, and per table after syncs that cascade on the server.

Table specs

Every table's behaviour is declared once in `TABLE_SPECS`. Adding a table means adding a spec, not writing another query.

```
const TABLE_SPECS: TableSpec[] = [
  { table: "profiles",      ownerColumn: "id",      reconcileDeletes: true },
  { table: "budgets",      ownerColumn: "user_id", reconcileDeletes: true },
  // ...categories, budget_items, assets, asset_categories, debts, reports,
  // merchant_rules, sms_blocklist: all full + delete sweep...
  { table: "sms_transactions", ownerColumn: "user_id",
    firstPull: { orderBy: "created_at", ascending: false, limit: 200 },
    reconcileDeletes: false },
  // asset_value_history (500), net_worth_snapshots (24), activity_logs (200),
  // feedback (100): truncated on first pull, never swept
];
```

Pulling one table

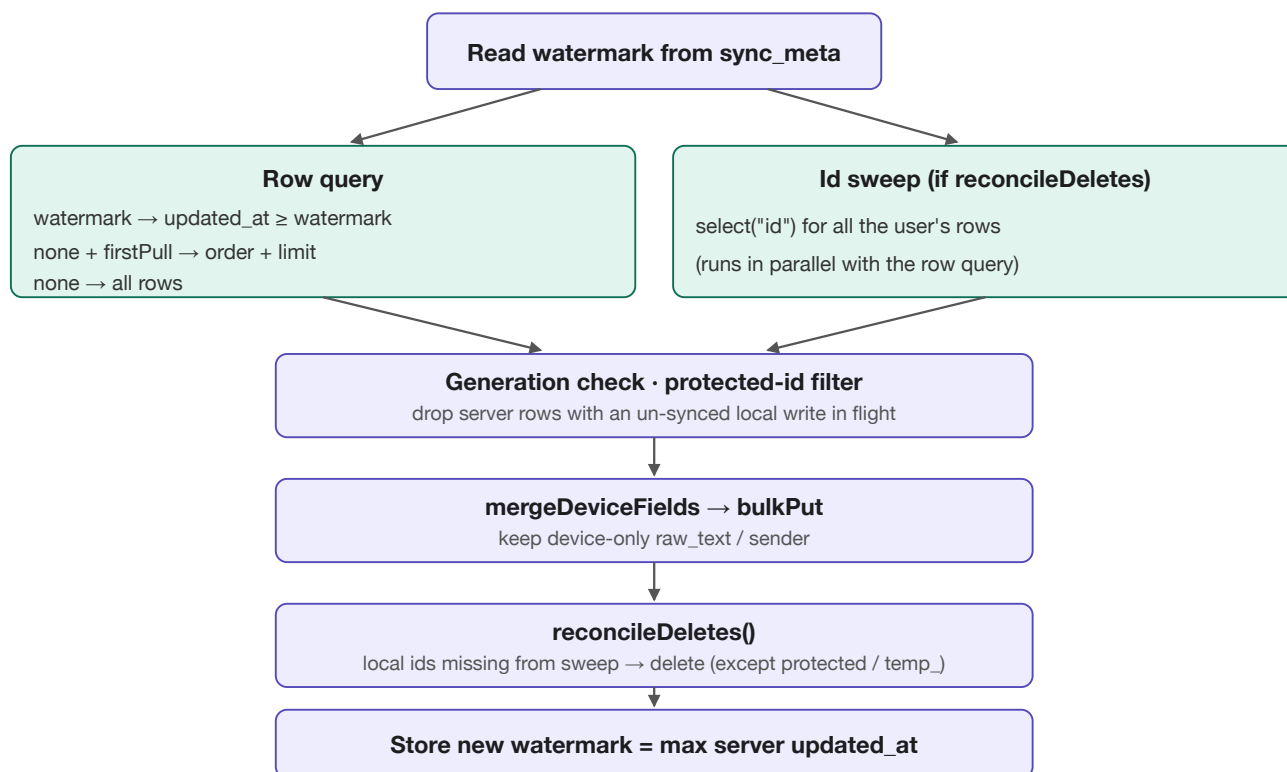


Figure 6. `pullTable()`. All 15 synced tables run in parallel; one failing table doesn't stop the others.

The rules that make delta sync safe

Rule	Why
Server timestamps only. The watermark is the newest server <code>updated_at</code> seen, never the client clock.	A phone clock running fast would otherwise skip rows written in between.
<code>gte</code> , not <code>gt</code> .	At microsecond resolution a tie could drop a row. Re-sending one row is harmless because <code>bulkPut</code> is idempotent.
Deletions come from the id sweep.	A delta can never mention a row that no longer exists.
Truncated tables are not swept.	For capped tables (sms_transactions, activity_logs, asset_value_history, net_worth_snapshots, feedback), a missing id just means "older than the cap".
Truncation applies to the first pull only.	Once a watermark exists, deltas are naturally small.
Degrades safely.	Without an <code>updated_at</code> column no watermark is stored, so an older database keeps doing full pulls. Clearing a watermark is the recovery path for any reconcile bug.
Failures are isolated.	A failed table keeps its old watermark and re-asks for the same window next time.

The server side of this is migration `20260912000000_delta_sync.sql`: every synced table has `updated_at` and a `BEFORE UPDATE` trigger (`set_updated_at()`), so every writer (server actions, SQL functions, the dashboard) bumps the watermark.

Protected ids: not clobbering local writes

If the user edited a row and that edit is still queued, the server's version is older than the local one.

`buildProtectedIds()` collects every `recordId` in pending or processing queue items (plus the real id of a temp id already mapped). Server rows with those ids are skipped. It runs **before and after** the fetch (`unionProtected`), which closes the race where an item drains while the fetch is in flight.

Wipe generation: never resurrect the previous user

```
let dbGeneration = 0;

export async function clearDB() {
  dbGeneration++;           // invalidate every pull already in flight
  // ...clear every table including sync_queue, id_map, sync_meta...
}

// inside hydrateAllTables / pullTable
const gen = dbGeneration;   // snapshot at start
const rows = await fetch(...);
if (isStale(gen)) return;   // user signed out meanwhile: discard payload
```

Without this, a slow hydrate that finished after sign-out would write the old user's rows into the freshly cleared database, and the next person to sign in on that device would see them.

Multi-account safety

`hydrateAllTables()` reads `sync_meta.__userId__`. If it differs from the signed-in user, IDB is wiped before pulling, then the new user id is stored.

12 SyncProvider: lifecycle and refresh

`lib/providers/SyncProvider.tsx` glues everything together. It creates the engine once, exposes `{pendingCount, isOnline, isHydrated, engine}` through context, and decides when to hydrate.

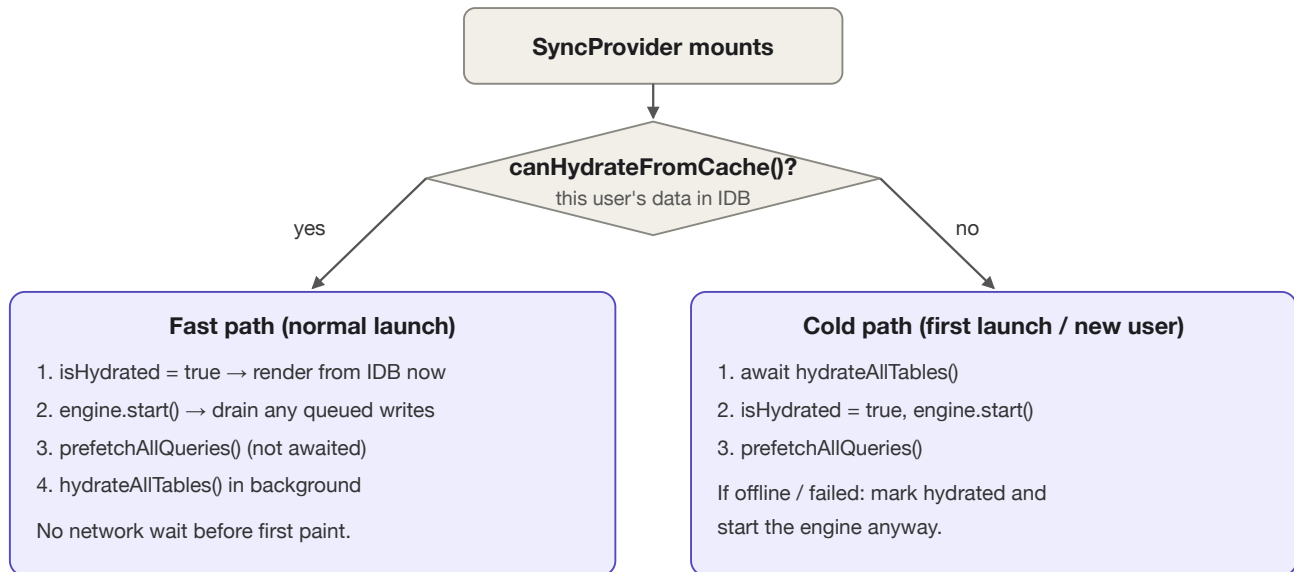


Figure 7. Startup. `isHydrated` gates things that need local data, like the SMS queue drain.

Foreground refresh

With `staleTime: Infinity`, cold-launch hydration would otherwise be the only server pull.

`refreshFromServer()` runs on `visibilitychange` (visible), `online`, and the Capacitor `resume` event. It is throttled:

- at most once per 20 seconds;
- skipped when `isTableStale("budgets")` is false (data under 5 minutes old);
- deferred with `requestIdleCallback` so a tab tap right after resume isn't starved;
- followed by a lazy `invalidateQueries({type: "active"})`, not a refetch storm.

Post-sync refresh map (`handleSynced`)

Synced item	Effect
budgets / categories / budget_items	Invalidate <code>budget</code> , <code>categoryData</code> , <code>categoryItems</code> , <code>dashboard</code>
budget_items UPDATE or PAYMENT	Re-pull assets, asset_value_history, debts (server cascade into linked asset or debt)
sms_transactions INSERT or CATEGORIZE	Re-pull budget_items, assets, debts (the server logged a spend)
budgets CARRY_SETUP	Re-pull budgets and budget_items (durable identity stamped server-side)
budgets UNDO_CARRY	Re-pull budgets, categories, budget_items with delete reconciliation
assets ACHIEVE	Re-pull assets, budget_items, net_worth_snapshots

assets / debts (any)	Invalidate <code>net-worth</code> , <code>goals</code> , <code>debt</code> , <code>dashboard</code>
----------------------	---

Daily activity stamp

Once per UTC day (gated by a localStorage day key), SyncProvider calls `touchLastSeen()` . It writes `profiles.last_seen_at` , `profiles.last_app_mode` (from `Capacitor.isNativePlatform()` , because the server cannot tell the WebView from a browser) and a row in `user_active_days` , which the admin portal uses for DAU, WAU and MAU.

13 Worked example: a budget item created offline

This timeline ties chapters 6 to 12 together. The user is offline, creates a category and an item inside it, then comes back online.

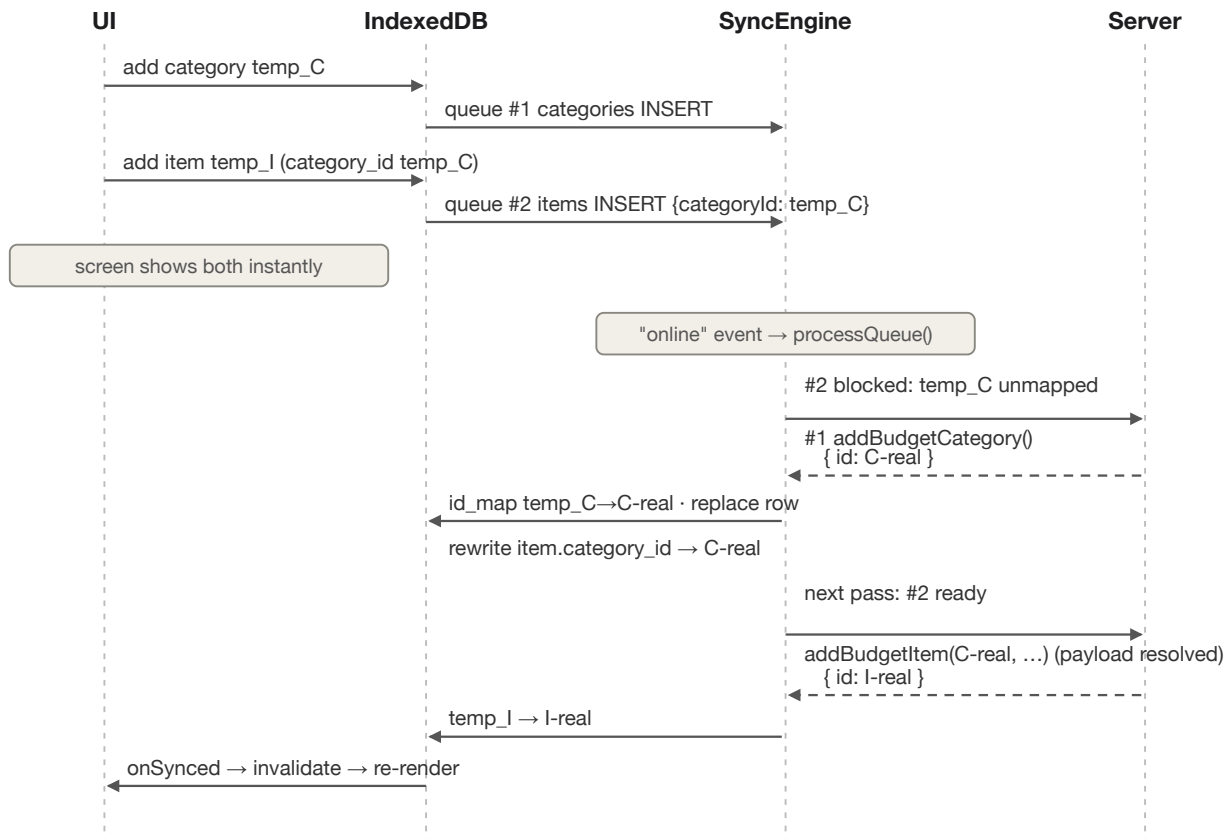


Figure 8. Dependent INSERTs. The item waits for its category; payload temp ids are rewritten before dispatch.

A detail page opened at `/budget/temp_C` keeps working after the swap, because `useCategoryData` resolves a temp id through `id_map`.

14 Server actions and cascades

Server actions in `lib/actions/<domain>.ts` are the only client-reachable path to Supabase for writes. They are called in two ways: by the SyncEngine when draining, and by read hooks on an IDB miss.

Anatomy of an action

```
"use server";

export async function addBudgetItem(categoryId, name, planned = 0, link, emoji = null, template) {
  const supabase = await createClient();           // request-scoped
  const user = await getAuthedUser();             // verified once per request
  if (!user) throw new Error("Unauthorized");

  // validate input + business rules (allocation can't exceed category)
  const ctx = await getItemAllocationContext(supabase, user.id, categoryId, null);
  validateItemAllocationChange(ctx.categoryAllocation, ctx.otherItemsPlanned, planned);
  if (link) await validateBudgetItemLink(supabase, user.id, link.link_type, link.link_id);

  const { data, error } = await supabase.from("budget_items")
    .insert({ category_id: categoryId, user_id: user.id, name: name.trim(), ... })
    .select().single();
  // ...logActivity(...), return full row (SyncEngine replaces the temp row with it)
}
```

Return the full row from INSERT actions. The SyncEngine writes whatever comes back into IDB. A partial object would wipe fields locally.

The spend cascade (`quickLogSpend`)

Logging a spend on an item is the busiest action. It is reached from quick spend (PAYMENT), SMS auto-allocation and SMS categorize.

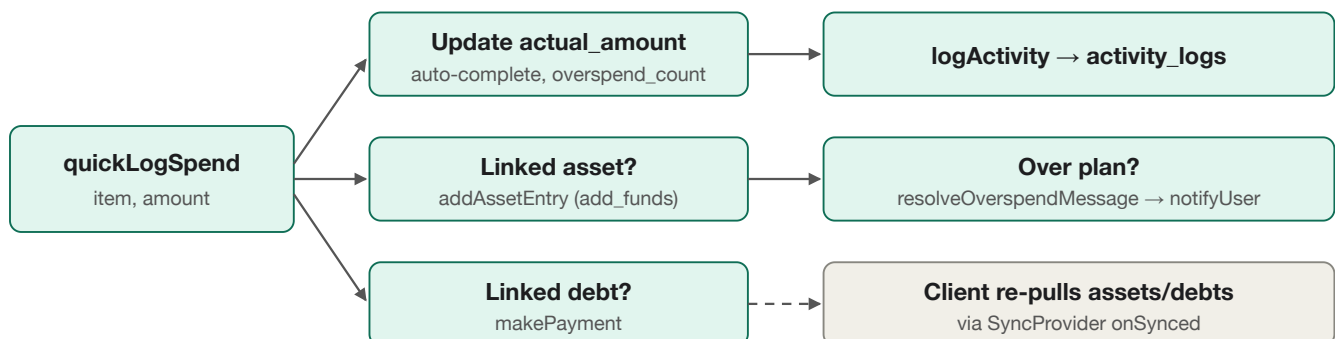


Figure 9. A spend can touch `budget_items`, `assets` or `debts`, `asset_value_history`, `activity_logs`, and push.

Budget items can link to an asset (e.g. "SIP" feeds a mutual fund) or a debt (e.g. "Car EMI" pays a loan). Spending on the item moves money into the asset or pays down the debt. `reverseSpend` undoes this when an SMS allocation is removed.

Activity log

`lib/server/activity-logger.ts` exposes `logActivity()` (writes `activity_logs` with category budget, net_worth, goals or debts), `fmt()` for money strings in the user's currency, and `getUserCurrency()`. Activity logs record *mutations* only, which is why a separate daily stamp exists for "active" users.

15 Database (Supabase / Postgres)

Core entity relationships

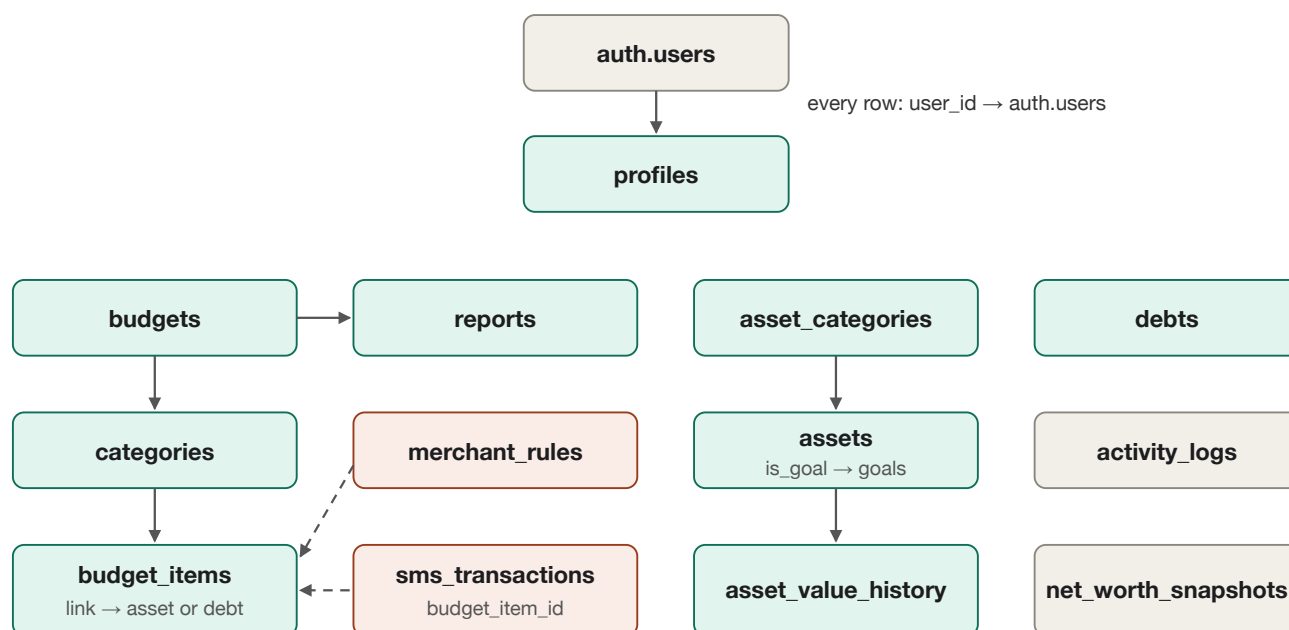


Figure 10. Every table carries `user_id` (FK to `auth.users`, on delete cascade). Arrows point parent → child; dashed lines are soft links (a transaction or rule points at a budget item; an item can link to an asset or a debt).

Security model

- **RLS everywhere.** Policies are uniformly "own row" (`user_id = auth.uid()`). No normal session can read another user's data.
- **Service-only tables.** `supporters`, `landing_events`, `play_install_stats` and `push_campaigns` have RLS on with zero policies, so only the service role can touch them.
- **Admin aggregates.** `admin_overview`, `admin_daily_series`, `admin_user_search`, `admin_user_detail`, `admin_feature_usage` and `admin_push_reach` are `security definer` functions with EXECUTE revoked from public, anon and authenticated.
- **Signup trigger.** `handle_new_user()` creates a profile row when an auth user is created.
- **AI quota.** The `increment_ai_usage` RPC counts messages per user per day.

Other tables

Table	Purpose
<code>budget_templates</code>	User-saved templates (categories as JSONB)
<code>sms_blocklist</code>	Hashed SMS template keys to skip
<code>feedback</code>	In-app bug reports and ideas
<code>push_subscriptions</code> / <code>fcm_tokens</code>	Web Push endpoints and Android FCM tokens (separate shapes)
<code>app_config</code>	Single row: minimum Android versionCode plus the <code>flags</code> JSONB

user_active_days	(user_id, day) primary key: real daily-active history
supporters	Ko-fi ledger keyed by email, idempotent on <code>last_message_id</code>
landing_events / play_install_stats / push_campaigns	Admin analytics and broadcast history

16 Android native shell

The Android app runs in **remote-URL WebView mode**. The native shell loads the deployed site, so SSR, server actions and the IDB layer work unchanged. No web assets are bundled (`webDir: "public"` is only a CLI formality).

```
// capacitor.config.ts
const config: CapacitorConfig = {
  appId: "com.octane.allocat",
  appName: "AlloCat",
  webDir: "public",
  server: {
    url: process.env.CAP_SERVER_URL || "https://allocat.xyz/dashboard",
    androidScheme: "https",
    cleartext: (process.env.CAP_SERVER_URL ?? "").startsWith("http://"),
  },
  plugins: { SplashScreen: { launchAutoHide: false, ... }, Keyboard: { resize: "none" }, ... },
};
```

How native and web work together

Concern	How it is handled
Splash screen	Auto-hide is off; <code>NativeShell</code> hides it once React mounts, so users never see a blank WebView.
Back button	<code>NativeShell</code> closes the topmost sheet first (vaul/Radix via synthetic Escape, custom sheets via <code>sheetRegistry</code>), then navigates.
OAuth	A custom-scheme deep link that the shell intercepts (AndroidManifest + NativeShell).
Platform detection	<code>Capacitor.isNativePlatform()</code> on the client. The server cannot tell the WebView from a browser.
Force update	<code>ForceUpdateGate</code> compares the installed <code>versionCode</code> with the minimum in <code>app_config</code> .
Android browser users	<code>AndroidWebGate</code> sends them to the Play Store (not shown inside the native app).
Keyboard	Resize is off; <code>KeyboardInset</code> publishes CSS variables that sheets use.
Haptics	<code>@capacitor/haptics</code> (the WebView lacks VIBRATE).
Local dev	<code>CAP_SERVER_URL=http://<LAN-IP>:3000 npx cap sync</code> . Needs JDK 21 (Android Studio JBR). <code>randomUUID</code> is polyfilled for the insecure http origin.

17 SMS transaction tracking

This is the core native feature: incoming bank and UPI debit SMS become budget spends automatically. The full design is in [docs/sms-feature.md](#).

Privacy and Play policy constraints

- Only `RECEIVE_SMS` is declared. The app never reads the existing inbox (`READ_SMS` is absent).
- `SmsFilter` drops anything that isn't financial-looking (and all OTPs) before it is queued or shown to JS.
- Parsing is on-device regex. There is no LLM or network parse (removed for compliance).
- Only extracted fields plus hashed keys sync. The raw SMS body and sender stay in the device's IDB and are protected from server overwrites (`lib/sync/deviceFields.ts`).
- Only debits are tracked; credits and ambiguous messages are ignored.

Two paths: app open vs app closed

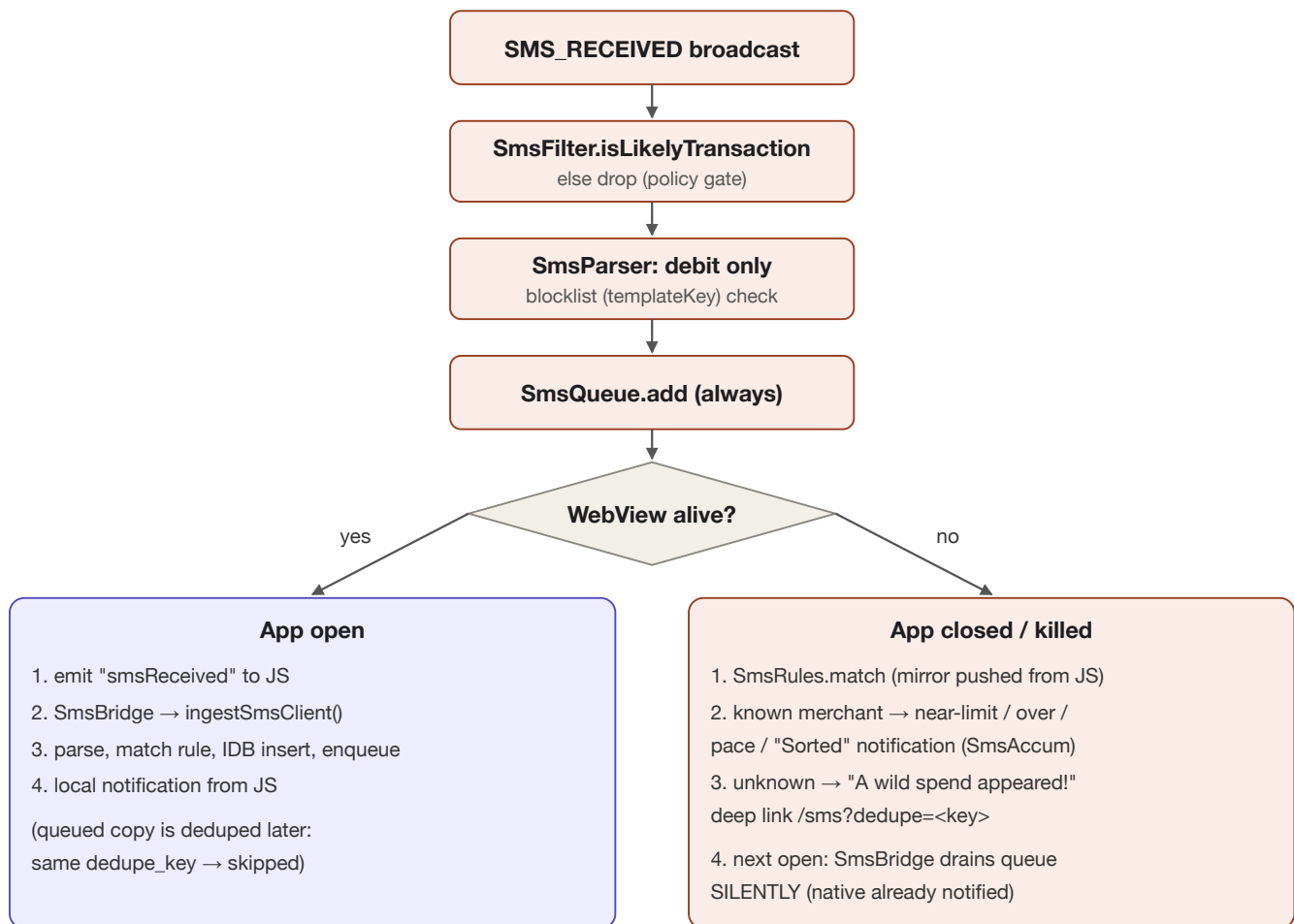


Figure 11. `SmsTransactionReceiver.java` runs even when the app is killed.

The client ingest pipeline (`lib/sms/ingestClient.ts`)

1. Trim. Drop OTP, verification and pre-auth messages (`is0tpOrVerification`).
2. Compute `dedupeKey = hash53(sender | normalized body)` and `templateKey = hash53(sender | skeleton)`. The skeleton masks every token containing a digit and every UPI handle.

3. Skip if the template is in `sms_blocklist`.
4. In-memory in-flight guard, then check IDB for the same `dedupe_key`. This matters because the live event and the queue drain can both deliver the same SMS.
5. `parseTransactionSms()`. Require an amount above 0 and `direction === "debit"`.
6. Match merchant rules (`exact > contains > regex`), then `selectRuleForPeriod()` resolves the rule to **this month's** budget item.
7. One Dexie `rw` transaction: re-check dedupe, insert the `temp_` transaction (status categorized or pending), and optimistically add the amount to the item's `actual_amount` (and `overspend_count`).
8. Enqueue `sms_transactions INSERT` with **only** amount, currency, merchantRaw, direction, occurredAt, dedupeKey and templateKey.
9. Notify: overspend (tiered message), near-limit (at least 90%), pace ("on track to run out around the 23rd"), or an optional "Sorted" confirmation. Every notification is recorded in the local inbox; the OS notification is suppressed when `silent`.

```
// what actually leaves the phone
await deps.enqueue({
  table: "sms_transactions", operation: "INSERT", recordId: tempId, tempId,
  payload: { amount, currency, merchantRaw: merchant, direction, occurredAt, dedupeKey,
    templateKey },
});
```

Server ingest (`lib/actions/sms.ts`)

`ingestOne()` is shared by the single and bulk entry points. It dedupes by `dedupe_key` (idempotent, returning the full existing row), checks the blocklist, requires an explicit debit, re-matches rules **server-side** (rules may have changed since capture), inserts the row, then, if a rule resolves for the period, calls `quickLogSpend`, marks the row categorized, bumps `times_applied` and checks the near-limit notification.

`ingestSmsTransactionsBulk` runs inputs **sequentially**, because two SMS may hit the same item and `quickLogSpend` is a read-modify-write.

Durable rules across months

Every month a template is applied, budget items get **new UUIDs**. A rule tied to last month's item id would break. So rules are keyed to a durable identity, `(template_id, template_item_id)`.

`lib/sms/resolveRuleItem.ts` maps that identity to the current period's item, and it is the single source of truth for both the client and server ingest paths.

Native mirror

The closed-app receiver can't query IDB. So `SmsBridge` pushes a snapshot to native on each foreground (`lib/sms/nativeMirror.ts`):

- `setRules` : rules resolved to current-month items, with planned, actual, category allocation and overspend count;
- `setQuickTargets` : top items shown as quick-allocate buttons;
- `setConfig` : notification sound and the auto-allocate confirmation toggle;
- `setBlocklist` : blocked template keys.

`SmsAccum` and `SmsOverspend` keep running totals across several closed-app spends until the next open refreshes the snapshot.

Keep in sync: `SmsParser.java` ↔ `lib/ai/parseSmsTransaction.ts`, `SmsHash.java` / `SmsSignature.java` ↔ `lib/sms/match.ts`, `SmsMessages.java` ↔ `lib/notify/messages.ts`. The hashes must be byte-for-byte compatible or deep links and blocklists silently stop matching.

User actions on /sms

Action	Queue op	Effect
Allocate a pending transaction	CATEGORIZE	Spend logged on the item; optionally learn a merchant rule
Recategorize / unallocate	RECATEGORY / UNALLOCATE	<code>reverseSpend</code> on the old item, then re-log
Ignore / delete	IGNORE / DELETE	Hidden or removed (reversing any spend)
Unblock a template	sms_blocklist DELETE	That kind of SMS is captured again
Not a transaction	sms_blocklist INSERT	Template key added to <code>sms_blocklist</code> ; future SMS of that kind are skipped on both paths

18 Notifications and push

There are three delivery channels. None of them reaches everyone, so each has a specific job.

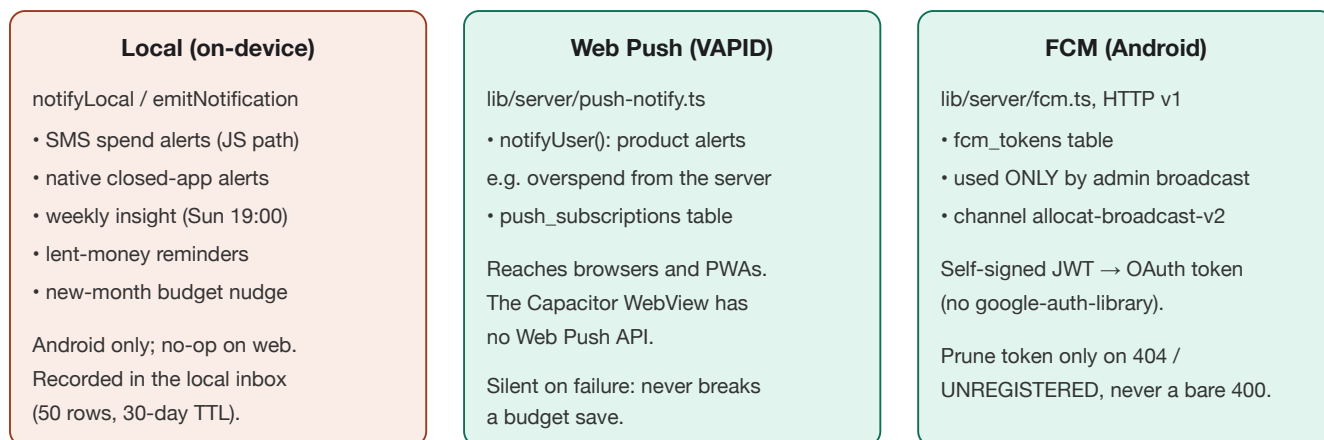


Figure 12. Notification transports.

Do not add FCM to `notifyUser`. SMS spend alerts are already raised on-device, while the server path calls `notifyUser` for the same event. Adding FCM there would double-notify every Android user. Broadcasts have no on-device twin, so they can safely use both transports.

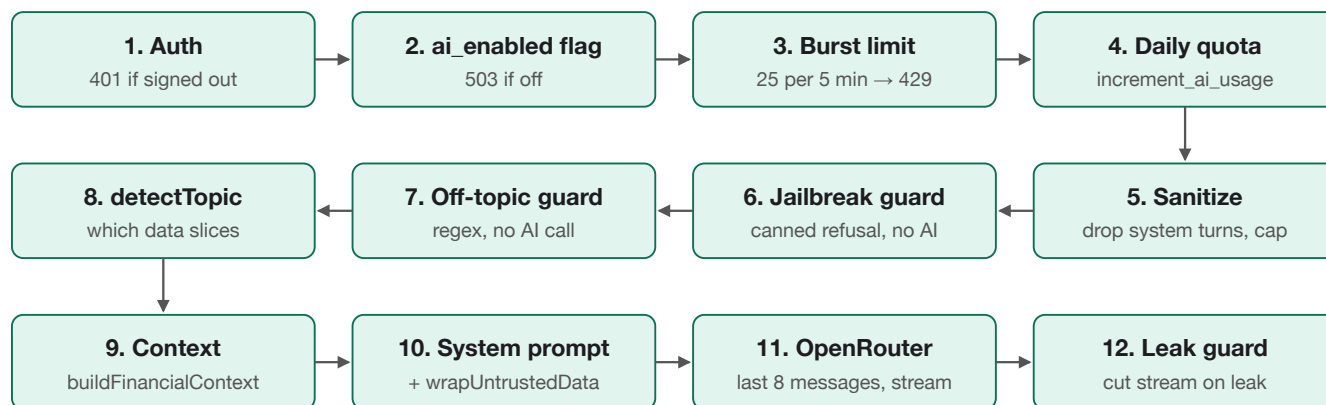
The FCM channel id `allocat-broadcast-v2` must match in three places: the manifest's `default_notification_channel_id`, `LocalNotifications.createChannel` in `PushRegistration.tsx`, and `android.notification.channel_id` in `fcm.ts`.

Message copy lives in `lib/notify/messages.ts`: cat-voice pools with escalating overspend tiers, picked by a stable seed so re-renders show the same variant. The server can replace the static text with an AI line (`resolveOverspendMessage`) and falls back to the pool on any failure.

19 AI features

All model calls go through `lib/server/openrouter.ts` (model `openrouter/free`). AI is free for everyone; the only ceiling is a daily message allowance.

Chat request pipeline (`app/api/ai/chat/route.ts`)



The cheap checks (flag, limits, regex guards) run before any data is fetched or any model is called.

The quota fails open: if the counter is unavailable, the chat goes through rather than locking a user out.

Figure 13. Chat pipeline. The response is Server-Sent Events.

Security layers (`lib/ai/guard.ts`)

The client posts the whole message history, so everything arriving is treated as attacker-controlled:

- `sanitizeMessages` drops client "system" turns, strips role spoofing and invisible characters, and caps lengths and counts.
- `isPromptExtraction` scans every user turn (attacks can be split across turns) and returns a fixed refusal.
- `wrapUntrustedData` fences the financial data (which includes SMS merchant text) so the model treats it as facts, not instructions.
- `createLeakGuardStream` watches the output and cuts the stream if the system prompt starts leaking.

Other AI uses

Feature	Files	Fallback
Weekly insight notification	<code>lib/actions/insights.ts</code> , <code>lib/ai/insightPrompt.ts</code> , <code>lib/sms/insightStats.ts</code> , <code>lib/sms/recap.ts</code>	Offline template (<code>pickFlavor</code>)
Monthly report summary	<code>lib/actions/monthly-summary.ts</code> , <code>lib/ai/monthlySummaryPrompt.ts</code>	Retry hint
Overspend message	<code>lib/actions/notify-messages.ts</code> , <code>lib/ai/overspendPrompt.ts</code>	Static message pool

Stats are computed on the client from IDB and sent as plain data; these actions never read the database themselves. Prompt builders and parsers are pure modules so they can be unit-tested.

20 Budget domain: templates, carry-forward, quiz

Structure

A **budget** is one month's total. It holds **categories** (type needs, wants, investments or misc, each with an allocation), and each category holds **items** (planned vs actual). An item is "done" automatically when actual reaches planned, and becomes not-done again if spending falls below (`computeAutoCompletion`). Allocations are validated so items can't exceed their category.

Templates

- Predefined templates live in `lib/budget-templates.ts` ; user templates are stored in `budget_templates` .
- Every template item has a **durable id**. When a template is applied, items are stamped with `template_id` and `template_item_id` . This is what lets SMS rules follow an item from month to month.
- `diffTemplateToBudget` applies template edits to the current budget (structural by default; amounts only when the user agrees). `templateDrift` detects when the budget no longer matches its template.
- Setup math (`lib/budget/setupMath.ts`) converts percentages to amounts and rebalances allocations.

Carry-forward

`CarryController` watches for an empty current month once IDB is hydrated. It copies the most recent prior budget (structure kept, spending reset) with a `CARRY_SETUP` queue op and shows a one-tap undo banner. The carry state is stored in `sync_meta` under `__carry__<year>--<month>` . Months other than the current one never auto-carry; their empty state offers "Copy from <month>". The pure logic is in `lib/budget/carry.ts` , shared by client and server.

Onboarding quiz

Five questions (habit, life, dream, treat, amount) are mapped by `lib/budget/quizMapping.ts` into a 3-category pseudo-template. The existing setup math turns it into real amounts. The user can save the plan or keep it for later (`quizDraft` , re-offered by the first-run checklist).

21 Admin portal

`/admin` is an internal insights and operations surface. It is deliberately simple: plain React Server Components and server actions, with **no React Query and no Dexie**.

- **Access:** `requireAdmin()` checks a confirmed email against the server-only `ADMIN_EMAILS` allowlist and calls `notFound()` otherwise, so non-admins get a 404, not a 403. It must be called at the top of **every** page, server action and route handler: server actions are addressable on their own, and a layout check does not protect them.
- **Reads:** `lib/admin/queries.ts` (`server-only`) uses the service client and the `admin_*` Postgres functions (supabase-js has no GROUP BY or COUNT DISTINCT).
- **Writes:** `lib/admin/actions.ts` : resolve feedback, link a supporter, send a test push, force sign-out, delete a user, save config, sync Play installs.
- **Pages:** overview, growth funnel, users (paged and sortable) and user detail, support (feedback plus the Ko-fi ledger), broadcast, config.
- **Charts:** two inline-SVG components (`Sparkline` , `BarSeries`). No chart library.
- **Active users:** DAU, WAU and MAU come from `user_active_days` , not from `last_seen_at` (a single overwritten column can't answer "how many were active on day X").
- **Play installs:** Play has no REST API for installs. `lib/play/installs.ts` reads bulk-report CSVs from the developer GCS bucket (UTF-16LE with a BOM), nightly through the Vercel cron.
- **Broadcast:** segments are all, android, web, supporters and inactive_7d. A test send to yourself is required first, and the real send requires typing the segment name. Counts are per device.

22 Runtime flags, force update, support

Runtime flags

Every `NEXT_PUBLIC_*` variable is baked in at build time. A kill switch has to work for Android builds that are already shipped, so flags live in `app_config.flags` (JSONB) and are served by `/api/app-config`.

```
// lib/config/flags.ts
export const DEFAULT_FLAGS: AppFlags = {
  ai_enabled: true,
  sms_enabled: true,
  support_cta_native: process.env.NEXT_PUBLIC_SUPPORT_CTA_NATIVE !== "false",
  daily_ai_messages: 30,
};
```

- Server: `getServerFlags()`, memoized 60s, fails open to defaults.
- Client: `useAppFlags()`, returns defaults until the fetch lands, refetched hourly. A flag can only switch something off deliberately, never as a side effect of being offline.
- `parseFlags` is tolerant: unknown keys are dropped and wrong types fall back to the default.

Force update

The same `app_config` row holds the minimum Android `versionCode`. `ForceUpdateGate` blocks older builds with a full-screen update prompt.

Support (no monetization)

- `/support` explains why AlloCat is free and links to Ko-fi. On native the link opens in the **system browser**, never an in-app checkout, and the button can be hidden with `support_cta_native` (an escape hatch for Play review).
- The Ko-fi webhook verifies the token, records the donation idempotently and sets `profiles.is_supporter`. `syncSupporterStatus()` links donations made before signup.
- `useIsSupporter()` is **cosmetic only** (a crown badge). Never gate a feature on it: that would turn an off-Play donation into a purchase of digital content.
- Any code mentioning premium, trial, entitlement or paywall is stale and should be removed.

23 PWA and service worker

AlloCat uses **Servist** (`@servist/next`), not next-pwa. `app/sw.ts` is compiled to `public/sw.js` by `servist build` after `next build`. The service worker is disabled in dev.

Request type	Strategy	Reason
Document navigation	NetworkFirst, 3s timeout, 32 entries, 24h	Fresh shell on launch, but a slow network can't hold the splash screen forever
RSC payloads and prefetch	StaleWhileRevalidate	Instant tab navigation. Safe because real data comes from IndexedDB, not the RSC payload.
Build assets	Precache (Servist manifest)	Offline shell
Offline	<code>/~offline</code> fallback	Friendly offline page

The manifest (`app/manifest.ts`) declares icons, shortcuts and a `share_target` : sharing text to AlloCat posts to `/share-target` , and `parseSpend()` pre-fills a quick spend.

24 UI system, theme and tour

- **Design system:** Neo · Lime. Tokens in `app/globals.css`, light and dark through `next-themes`.
- **Accents:** lime (default), tangerine, lemon, purple, blue (`lib/theme/accents.ts`). `data-accent` on `<html>` swaps tokens. `AccentProvider` persists the choice, and a blocking script in the root layout applies it before first paint.
- **Fonts:** Hanken Grotesk (`--font-sans`), Bricolage Grotesque (`--font-display`), JetBrains Mono (`--font-mono`). Material Symbols is self-hosted (the CDN version raced first paint in the WebView).
- **Charts:** categorical palette keys `cat-1..cat-8` are stored on entities, never raw hex (`lib/theme/dataViz.ts`).
- **Sheets:** vault drawers with custom keyboard handling (`KeyboardInset`) and scroll lock (`SheetScrollLock`).
- **Currency:** multi-currency through `lib/number-format.ts` and `lib/currency/catalog.ts`. INR is only a legacy default; never hardcode `en-IN`.
- **Tour:** `driver.js`, managed by `lib/tour/`. `TourContext` persists `{enabled, seenPages}` in `localStorage` (`allocat-tour-state`). It is opt-in through `TourPrompt`, and pages show mock data while touring. To add a page, extend `tourSteps.ts` and `types.ts`.

25 Testing, tooling and environment

Commands

```
pnpm install                # npm install fails (ERESOLVE)
npm run dev                 # next dev
npm run build               # next build && serwist build
npm run lint                # eslint (flat config)
npm run test                # vitest run
npx vitest run lib/sms/match.test.ts
npx vitest run -t "matches exact rule"
npx tsc --noEmit            # typecheck (no script)
npx cap sync android        # copy config + plugins into android/
CAP_SERVER_URL=http://192.168.1.20:3000 npx cap sync
```

Tests

Tests live next to their source. The heaviest coverage is on the riskiest logic: `SyncEngine.test.ts` (ordering, batching, collapse), the `hydrate*.test.ts` files (delta, generation, deletes), `ingestClient.test.ts` (duplicate races), `match`, `resolveRuleItem`, `nativeMirror`, `reverse-spend`, `carry`, `setupMath`, `quizMapping`, the AI guard and prompt parsers, and the Play CSV parser. There is no fake IndexedDB or Supabase library; tests inject small in-memory stubs.

Environment variables

Variable	Used for
<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_DEFAULT_KEY</code>	Supabase clients
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Service client: push, AI counter, Ko-fi, admin
<code>OPENROUTER_API_KEY</code>	AI chat and AI notifications
<code>KOFI_VERIFICATION_TOKEN</code> , <code>NEXT_PUBLIC_KOFI_URL</code>	Webhook (503 without it) and the support link
<code>ADMIN_EMAILS</code>	Admin allowlist (unset means /admin always 404s)
<code>CRON_SECRET</code>	Bearer token for the Play sync cron
<code>PLAY_SA_JSON_B64</code> , <code>PLAY_BUCKET</code>	Play install stats
<code>VAPID_PUBLIC_KEY</code> , <code>VAPID_PRIVATE_KEY</code> , <code>VAPID_SUBJECT</code>	Web Push
<code>FCM_PROJECT_ID</code> , <code>FCM_SA_JSON_B64</code>	Android push
<code>NEXT_PUBLIC_SUPPORT_CTA_NATIVE</code>	Build-time default for the <code>support_cta_native</code> flag

`CAP_SERVER_URL`

Point the Android shell at a LAN dev
server

26 Rules of thumb for contributors

When you...	Remember to...
Add a synced table	Add a Dexie version block, a <code>TABLE_SPECS</code> entry, <code>updated_at</code> + trigger in SQL, RLS policies, dispatch entries, and a <code>clearDB</code> line.
Add a queue operation	Register it in the SyncEngine <code>dispatch</code> table. Use <code>temp_</code> ids in payloads so <code>extractTempIds</code> can gate dependencies. Return the full row from INSERT actions.
Add an UPDATE payload	Only put the table in <code>COLLAPSIBLE_UPDATE_TABLES</code> if the write is absolute (not a delta).
Make an op batchable	Write a bulk action returning positional outcomes, and add a <code>bulkDispatch</code> entry.
Write a server action	Use <code>getAuthedUser()</code> , validate ownership (<code>user_id</code>), and keep it in <code>lib/actions/<domain>.ts</code> .
Write a read hook	Read IDB first, fall back to the server, and export the query key.
Prefetch a key	Use <code>fetchQuery</code> , not <code>prefetchQuery</code> .
Add a protected route	Decide whether it belongs in the middleware list (goals, profile and activity are not).
Touch SMS parsing or hashing	Change the Java twin in the same commit.
Add admin functionality	Call <code>requireAdmin()</code> in every entry point; keep React Query and Dexie out.
Need a kill switch	Add a runtime flag, not a <code>NEXT_PUBLIC_*</code> variable.
Format money	Use <code>useFormatCurrency</code> / <code>formatCurrency</code> ; never hardcode INR or <code>en-IN</code> .
Send a notification from the server	Use <code>notifyUser</code> (Web Push only). Never add FCM to it.

27 File-by-file reference

A short description of every source file, grouped by folder. Tests (`*.test.ts`) sit next to the module they cover and are listed only where they add context.

Root configuration

Build, tooling and platform config at the repo root.

<code>package.json</code>	npm manifest (name AlloCat-web). Scripts: dev, build (next build && servist build), start, lint, test. Declares Next 16, React 19, Supabase, Dexie, TanStack Query, Capacitor 8, Servist, vault, motion, driver.js, web-push.
<code>pnpm-lock.yaml</code> / <code>package-lock.json</code>	Lockfiles. pnpm is the working package manager (npm install fails with ERESOLVE).
<code>proxy.ts</code>	Next 16 middleware entry (named proxy, not middleware). Calls <code>updateSession()</code> . The matcher excludes /api, static assets, the service worker and images, because every matched request costs a Supabase Auth round trip.
<code>next.config.ts</code>	Next config wrapped for Servist. Allows private-LAN origins in dev so the Android shell can load a dev server over <code>http://<LAN-IP>:3000</code> .
<code>capacitor.config.ts</code>	Capacitor config. <code>appId</code> <code>com.octane.allocat</code> . Remote-URL mode: the WebView loads <code>https://allocat.xyz/dashboard</code> (or <code>CAP_SERVER_URL</code>). Configures SystemBars, LocalNotifications icon, Keyboard (no resize) and a manually hidden SplashScreen.
<code>servist.config.js</code>	Servist build config: compiles <code>app/sw.ts</code> into <code>public/sw.js</code> .
<code>vercel.json</code>	Vercel cron: GET /api/admin/cron/play-sync every day at 06:00 UTC.
<code>tsconfig.json</code>	TypeScript config. Path alias <code>@/*</code> maps to the repo root.
<code>eslint.config.mjs</code>	ESLint flat config extending <code>eslint-config-next</code> , with custom ignore overrides.
<code>postcss.config.mjs</code>	PostCSS with the <code>@tailwindcss/postcss</code> plugin (Tailwind v4, no <code>tailwind.config</code> file).
<code>vitest.config.ts</code>	Vitest config. Resolves the <code>@/*</code> alias so tests can import internal modules.
<code>CLAUDE.md</code> / <code>README.md</code> / <code>LICENSE</code>	Contributor guidance for AI agents, project readme and license.
<code>scripts/diagnose-play-access.mjs</code>	CLI check for whether the Google Play bulk-report bucket grant is live. It lists the bucket, because GCS answers 403 both for missing permission and for a missing object.
<code>play-assets/README.md</code>	Notes on the Play Store listing assets.

app/ — root, PWA and misc routes

App Router entry points outside the protected shell.

<code>app/layout.tsx</code>	Root HTML layout. Loads fonts (Hanken Grotesk, Bricolage Grotesque, JetBrains Mono), preloads the self-hosted Material Symbols font, and runs a blocking script that applies the saved accent before first paint. Provider order: ThemeProvider > AccentProvider > QueryProvider, plus RegisterPWA, NativeShell, AndroidWebGate and ForceUpdateGate.
-----------------------------	--

<code>app/globals.css</code>	Design tokens for the Neo · Lime system: light/dark colors, [data-accent] overrides, data-viz palette --cat-1..8, the @font-face for Material Symbols, and sheet and utility classes.
<code>app/page.tsx</code>	Root / route. Redirects into the app, or shows marketing metadata.
<code>app/manifest.ts</code>	PWA web manifest: icons, theme, shortcuts, and a share_target that POSTs to /share-target.
<code>app/sw.ts</code>	Service worker source (Serwist). Precache plus runtime caching. Document navigations are NetworkFirst with a 3s timeout; RSC payloads are StaleWhileRevalidate so tab navigation is instant, which is safe because real data comes from IndexedDB.
<code>app/~offline/page.tsx</code>	Offline fallback page served by the service worker.
<code>app/robots.ts</code> / <code>app/sitemap.ts</code>	SEO robots and sitemap generators.
<code>app/share-target/route.ts</code>	Web Share Target endpoint. Receives shared text from the OS share sheet and redirects to the dashboard with it, where parseSpend() pre-fills a quick spend.
<code>app/onboarding/page.tsx</code>	Post-signup onboarding: teaching slides deck, then the budget quiz. Warms the dashboard route and data so landing afterwards has no skeleton.
<code>app/legal/privacy-policy/page.tsx</code>	Privacy policy page (required for Play SMS permission review).
<code>app/legal/delete-account/page.tsx</code>	Public account-deletion instructions page (Play requirement).

app/auth/ — authentication

Login, signup and the OAuth callback.

<code>app/auth/login/page.tsx</code>	Email/password login form in the AuthShell. Calls the login server action; OAuth buttons alongside.
<code>app/auth/signup/page.tsx</code>	Signup form. Calls the signup server action.
<code>app/auth/callback/route.ts</code>	OAuth / email-confirm callback. Exchanges the code for a session cookie, then redirects (onboarding or dashboard), or to an error page.

app/(app)/ — protected app shell

Every signed-in screen. The layout mounts all providers and background controllers; each page is a thin wrapper around a components/* page.

<code>app/(app)/layout.tsx</code>	Shell layout. Provider nesting: TourProvider > SyncProvider > CurrencyProvider > QuickActionProvider. 480px mobile frame and md: desktop sidebar. Mounts PullToRefresh, PagerGate, InstallPrompt, KeyboardInset, SmsBridgeGate, PushRegistration, SheetScrollLock, TourPrompt, NativeSetup, AppBadgeReset, SupporterSync, CarryController, BudgetReminderController and BottomDock.
<code>app/(app)/loading.tsx</code>	Route-level loading skeleton.
<code>app/(app)/dashboard/page.tsx</code>	Dashboard route. Shows the first-run checklist alone while the setup guide is active, otherwise DashboardPage.
<code>app/(app)/budget/page.tsx</code>	Budget route: month budget, categories, setup sheet.
<code>app/(app)/budget/[categoryId]/page.tsx</code>	Category detail route (items inside a category).
<code>app/(app)/debt/page.tsx</code>	Debts and money-lent route.
<code>app/(app)/goals/page.tsx</code>	Savings goals route (goals are assets with is_goal = true).

<code>app/(app)/net-worth/page.tsx</code>	Net worth route: assets, categories, history.
<code>app/(app)/activity/page.tsx</code>	Activity log route.
<code>app/(app)/notifications/page.tsx</code>	Device-local notification inbox route.
<code>app/(app)/profile/page.tsx</code>	Profile and settings route.
<code>app/(app)/reports/page.tsx</code>	Monthly report route.
<code>app/(app)/sms/page.tsx</code>	SMS transactions route (pending / allocated).
<code>app/(app)/support/page.tsx</code>	"Why AlloCat is free" support page.
<code>app/(app)/transactions/page.tsx</code>	Unified transaction list route (SMS plus manual).

app/admin/ — internal admin portal

Plain RSC plus server actions. No React Query and no Dexie. Every entry point calls `requireAdmin()`.

<code>app/admin/layout.tsx</code>	Admin shell. <code>force-dynamic</code> (never cached or prerendered), <code>AdminNav</code> , and the <code>requireAdmin()</code> gate.
<code>app/admin/page.tsx</code>	Overview: DAU/WAU/MAU tiles, signups, an action strip, sparklines.
<code>app/admin/growth/page.tsx</code>	Growth funnel: landing views > Play clicks > installs > signups > onboarded > active. Plus Play install bars.
<code>app/admin/users/page.tsx</code>	Paginated, sortable user table (<code>admin_user_search</code> RPC).
<code>app/admin/users/[id]/page.tsx</code>	One user's detail (<code>admin_user_detail</code>) plus <code>UserActions</code> (delete, force sign-out, test push).
<code>app/admin/support/page.tsx</code>	Feedback inbox plus the Ko-fi supporters ledger.
<code>app/admin/broadcast/page.tsx</code>	Compose and send push broadcasts to segments.
<code>app/admin/config/page.tsx</code>	Edit runtime flags and the force-update minimum versions.

app/api/ — route handlers

HTTP endpoints. The middleware skips `/api`, so each route authenticates itself.

<code>app/api/ai/chat/route.ts</code>	Streaming AI chat. Order: auth > ai_enabled flag > burst rate limit > daily quota (<code>increment_ai_usage</code>) > sanitize history > jailbreak guard > off-topic guard > topic-scoped context > system prompt > OpenRouter stream > leak-guard stream.
<code>app/api/app-config/route.ts</code>	Public, uncached GET: force-update minimum versions plus runtime flags.
<code>app/api/kofi/webhook/route.ts</code>	Ko-fi donation webhook. Verifies <code>verification_token</code> (constant-time), upserts supporters idempotently on <code>message_id</code> , and flips <code>profiles.is_supporter</code> . Returns 503 when unconfigured.
<code>app/api/track/route.ts</code>	Anonymous landing-page beacon. Stores event name, coarse platform, path and referrer hostname. No IP, no user agent, no id. Event names are allowlisted; rate-limited in memory.
<code>app/api/admin/broadcast/route.ts</code>	Admin push broadcast endpoint (GET reach counts, POST send). <code>requireAdmin</code> plus a long <code>maxDuration</code> .
<code>app/api/admin/cron/play-sync/route.ts</code>	Nightly cron (<code>CRON_SECRET</code> bearer) that mirrors Play install CSVs into <code>play_install_stats</code> .

components/ — shell and navigation

App-wide chrome.

<code>components/BottomDock.tsx</code>	Mobile bottom dock: nav pill plus an optional route-aware quick-action button, animated as one group.
<code>components/BottomNav.tsx</code>	Glass bottom nav with tap and scrub-slide gestures between tabs.
<code>components/SidebarNav.tsx</code>	Desktop (md:) sidebar navigation.
<code>components/QuickActionDock.tsx</code>	The floating paw quick-action button (CSS-mask icon).
<code>components/PullToRefresh.tsx</code>	Pull-to-refresh gesture with a Lottie paw. Triggers <code>hydrateAllTables</code> and a query refresh.

components/pager/

Native swipe pager.

<code>components/pager/PagerGate.tsx</code>	On native at one of the 5 tab roots, renders <code>TabPager</code> instead of children (avoids double-mount). Everywhere else renders children.
<code>components/pager/TabPager.tsx</code>	Horizontal swipe pager holding all 5 tab screens on native. Order must match <code>navItems</code> .

components/pwa/ — platform glue

PWA and Capacitor integration components.

<code>components/pwa/NativeShell.tsx</code>	Native-only setup: hides the splash, handles the Android back button (closes sheets first), OAuth deep links, status bar, and a rate-limited feedback prompt.
<code>components/pwa/NativeSetup.tsx</code>	First-run Android setup: asks for notification and SMS permission, then a guide for OEM autostart and battery exemption.
<code>components/pwa/SmsBridge.tsx</code>	Native bridge. Live <code>smsReceived</code> events go to <code>ingestSmsClient</code> (with notification). Queued SMS are drained silently after hydration. Also mirrors rules, targets, config and blocklist to native and follows notification deep links.
<code>components/pwa/PushRegistration.tsx</code>	Native-only FCM token registration into <code>fcm_tokens</code> . Creates the <code>allocat-broadcast-v2</code> notification channel.
<code>components/pwa/PushPermissionPrompt.tsx</code>	Web Push opt-in prompt (VAPID subscription into <code>push_subscriptions</code>).
<code>components/pwa/ForceUpdateGate.tsx</code>	Blocks an outdated native build: compares <code>App.getInfo().build</code> with the minimum from <code>/api/app-config</code> .
<code>components/pwa/AndroidWebGate.tsx</code>	In an Android browser (not the native app), gates the web app and points to the Play Store.
<code>components/pwa/KeyboardInset.tsx</code>	Publishes <code>--app-vh</code> and <code>keyboard-inset</code> CSS variables so bottom sheets sit correctly above the soft keyboard.
<code>components/pwa/SheetScrollLock.tsx</code>	Locks document scroll and disables the dock while any vault sheet is open (vault only locks on iOS Safari).
<code>components/pwa/AppBadgeReset.tsx</code>	Clears the OS app-icon badge once per launch.

components/ui/ — design-system primitives

Reusable building blocks.

<code>components/ui/Button.tsx</code>	Button with variants and sizes (md/lg meet the 44px tap target).
<code>components/ui/IconButton.tsx</code>	Icon-only button with a required aria label.
<code>components/ui/Card.tsx / Chip.tsx</code>	Card surface and chip/pill primitives.
<code>components/ui/HeroCard.tsx</code>	The lime hero card: paw watermark, big figure, progress and CTA.
<code>components/ui/StatCard.tsx</code>	Health-app-style stat pair card (ink or white tone).
<code>components/ui/Progress.tsx</code>	Unified progress indicator: ticks, segments or solid, with normal/warn/over states.
<code>components/ui/Field.tsx</code>	Label, input and error/hint wired together with a generated id.
<code>components/ui/InlineEditableNumber.tsx / InlineEditableText.tsx</code>	Tap-to-edit inline values.
<code>components/ui/SegmentedControl.tsx</code>	Underline tabs or pill filter segmented control.
<code>components/ui/BottomSheetSelect.tsx</code>	Select control rendered as a bottom sheet.
<code>components/ui/drawer.tsx</code>	Wrapper around vault Drawer that disables vault's buggy keyboard handling in favor of KeyboardInset.
<code>components/ui/ConfirmDrawer.tsx</code>	Confirm/cancel bottom sheet for destructive actions.
<code>components/ui/SwipeToDeleteRow.tsx</code>	Swipe-left-to-reveal-delete list row.
<code>components/ui/EmojiPickerModal.tsx</code>	Emoji picker portal (can mount inside a vault drawer).
<code>components/ui/ColorPicker.tsx</code>	Palette-key picker (cat-1..cat-8).
<code>components/ui/CurrencyText.tsx / CurrencySymbol.tsx</code>	Currency-aware amount and symbol rendering.
<code>components/ui/EmptyState.tsx / Skeleton.tsx</code>	Empty-state block and loading skeleton bars.
<code>components/ui/CrownBadge.tsx</code>	Lottie crown shown for supporters (cosmetic only).
<code>components/ui/InstallPrompt.tsx / RegisterPWA.tsx</code>	PWA install prompt UI and service worker registration.
<code>components/ui/MaterialSymbol.tsx</code>	Material Symbols icon component.
<code>components/ui/charts/DonutChart.tsx</code>	Donut chart used by the budget and net worth pages.

components/budget/

Monthly budget UI.

<code>components/budget/BudgetPage.tsx</code>	Budget screen: month picker (12 back, 3 forward), hero total, category list, template header and drift, setup entry.
<code>components/budget/BudgetSetupSheet.tsx</code>	The big setup sheet. Modes: create, saveTemplate, editTemplate, updateTemplate, editLinkedTemplate. Builds categories and items from templates or by hand.
<code>components/budget/BudgetQuickSetup.tsx</code>	Quick first-budget setup (pre-filled from a saved quiz draft).
<code>components/budget/BudgetEmptyState.tsx</code>	Empty month. The CTA becomes "Copy from <month>" when a carry source exists.
<code>components/budget/CategoryDetailPage.tsx</code>	Category detail: items list, add/edit items, allocation math, links to assets and debts.

<code>components/budget/ItemDetailSheet.tsx</code>	Create/edit a budget item: name, emoji, planned amount, link to an asset or debt, name suggestions.
<code>components/budget/ItemTransactionList.tsx</code>	Transactions allocated to one item.
<code>components/budget/CarryController.tsx</code>	Global auto-carry: once hydrated, copies the latest prior budget into an empty current month, with an undo banner.
<code>components/budget/BudgetReminderController.tsx</code>	Native new-month nudge when there is no budget and nothing to carry.

components/dashboard/

Home screen.

<code>components/dashboard/DashboardPage.tsx</code>	Dashboard: greeting, budget hero, quick spend, top spending, goals glimpse, support nudge.
<code>components/dashboard/QuickSpendInput.tsx</code>	Quick log-a-spend input with a category/item target picker drawer (quickLogSpend via PAYMENT).
<code>components/dashboard/TopSpendingGlimpse.tsx</code>	Top items by spend, warning tint over ~90%.
<code>components/dashboard/FirstRunChecklist.tsx</code>	First-run setup checklist (budget, goals, etc.) and the useFirstRun state.

components/debt/

Debts and money lent.

<code>components/debt/DebtPage.tsx</code>	Debt screen: owed vs lent tabs, totals, list.
<code>components/debt/DebtDetailSheet.tsx</code>	Create/edit a debt (flat or diminishing interest, tenure, EMI).
<code>components/debt/DebtPaymentSheet.tsx</code>	Record a payment against a debt.
<code>components/debt/DebtPaymentHistory.tsx</code>	Per-debt payment log derived from activity_logs.
<code>components/debt/LentListView.tsx</code>	Money-lent list, detail sheet and repayment sheet.
<code>components/debt/DebtEmptyState.tsx</code>	Empty state.

components/net-worth/ and goals/

Assets, net worth and savings goals.

<code>components/net-worth/NetWorthPage.tsx</code>	Net worth: assets minus liabilities, pie by category, monthly history.
<code>components/net-worth/AddAssetSheet.tsx</code>	Add an asset (with new-category creation).
<code>components/net-worth/AssetDetailSheet.tsx</code>	Asset detail and edit.
<code>components/net-worth/AssetEntrySheet.tsx</code>	Add funds, withdraw or update value (writes asset_value_history).
<code>components/net-worth/NetWorthEmptyState.tsx</code>	Empty state.

<code>components/goals/GoalsPage.tsx</code>	Goals list: progress, quick update, achieve.
<code>components/goals/GoalDetailSheet.tsx</code>	Create/edit a goal (an asset with a target).

components/sms/ and transactions/

SMS allocation UI.

<code>components/sms/SmsPage.tsx</code>	Pending vs allocated SMS transactions: allocate, rename, recategorize, ignore, delete, "not a transaction" (blocklist). Handles ?dedupe= deep links.
<code>components/sms/AllocateSheet.tsx</code>	Picker to allocate a transaction to a budget item (optionally creating a rule).
<code>components/transactions/TransactionsPage.tsx</code>	Unified list of SMS and manual transactions with item/category name lookup from IDB.

components/ai/

In-app AI chat.

<code>components/ai/AIOverlay.tsx</code>	Lazy-loads the chat drawer.
<code>components/ai/ChatDrawer.tsx</code>	Chat UI. Streams /api/ai/chat SSE and shows a plain message for the daily limit.
<code>components/ai/ChatInput.tsx</code> / <code>ChatMessage.tsx</code>	Input box and message bubbles (markdown).
<code>components/ai/SuggestedPrompts.tsx</code> / <code>LoadingQuote.tsx</code> / <code>PawLogo.tsx</code>	Starter prompts, a loading quote and the paw logo.

components/onboarding/

Onboarding deck and budget quiz.

<code>components/onboarding/OnboardingDeck.tsx</code> / <code>OnboardingCard.tsx</code> / <code>onboardingSlides.tsx</code>	Swipeable teaching slides with mini feature previews.
<code>components/onboarding/AnimatedNumber.tsx</code>	Count-up number animation.
<code>components/onboarding/quiz/QuizFlow.tsx</code>	Five-question quiz flow ending in RevealScreen.
<code>components/onboarding/quiz/useQuizAnswers.ts</code>	Quiz answer reducer (skip-safe defaults).
<code>components/onboarding/quiz/QuizChip.tsx</code> / <code>QuizOptionCard.tsx</code> / <code>QuizQuestionShell.tsx</code>	Quiz UI primitives.
<code>components/onboarding/quiz/screens/Q1Habit..Q5Amount.tsx</code>	The five questions: habit, life, dream, treat, amount.
<code>components/onboarding/quiz/screens/RevealScreen.tsx</code>	Shows the generated plan. Save it, or keep it for later (saved as a quiz draft).

components/ — other features

Profile, support, activity, notifications, tour, auth, admin, feedback.

<code>components/profile/ProfilePage.tsx</code>	Profile and settings: avatar, currency, accent, sound, notification toggles, support, logout, delete account.
<code>components/profile/AvatarPickerSheet.tsx</code> / <code>UserAvatar.tsx</code>	Pick and render one of 12 cat avatars.
<code>components/profile/CurrencySelector.tsx</code> / <code>AccentSelector.tsx</code> / <code>NotificationSoundSelector.tsx</code>	Settings selectors.
<code>components/profile/SupportCard.tsx</code>	Quiet entry point to the support page.
<code>components/support/SupportPage.tsx</code>	Explains why AlloCat is free. Ko-fi link opens in the system browser. Hidden on native when <code>support_cta_native</code> is off.
<code>components/support/SupportNudge.tsx</code>	One-time dashboard nudge after the account is 14 days old. Never shown to supporters.
<code>components/support/SupporterSync.tsx</code>	Calls <code>syncSupporterStatus()</code> once per day.
<code>components/activity/ActivityPage.tsx</code> / <code>ActivityLogItem.tsx</code> / <code>ActivityDetailSheet.tsx</code>	Activity log list, row and detail.
<code>components/notifications/NotificationsPage.tsx</code> / <code>NotificationBell.tsx</code>	Local notification inbox and unread badge.
<code>components/tour/TourPrompt.tsx</code> / <code>TourBanner.tsx</code>	Opt-in prompt for the driver.js tour, and the in-tour banner.
<code>components/auth/AuthShell.tsx</code> / <code>OAuthButtons.tsx</code>	Auth page frame and OAuth buttons (native uses a custom-scheme deep link).
<code>components/feedback/FeedbackSheet.tsx</code>	Bug, idea and feedback form (online-only <code>submitFeedback</code>).
<code>components/admin/*</code>	Admin UI: AdminNav, Tile, BroadcastForm (a test send is required first; type the segment name to confirm), ConfigForm, FeedbackList, SupporterList, UserActions, SyncInstallsButton, charts/Sparkline and charts/BarSeries (inline SVG, no chart library).

lib/db/ — IndexedDB layer

The local database that every screen reads first.

<code>lib/db/AllocatDB.ts</code>	Dexie schema (version 19). Mirror tables plus the local-only notifications table and the sync infra tables <code>sync_queue</code> , <code>id_map</code> , <code>sync_meta</code> . Row types and the <code>SyncTable</code> , <code>SyncOperation</code> , <code>SyncStatus</code> , <code>SyncQueueItem</code> types.
<code>lib/db/index.ts</code>	<code>getDB()</code> : a browser-only singleton (throws on the server).
<code>lib/db/hydrate.ts</code>	Server-to-IDB reconciliation: <code>TABLE_SPECS</code> , delta pulls with a watermark, id-sweep delete reconciliation, protected ids for in-flight writes, device-field merge, the wipe-generation guard, <code>clearDB()</code> , <code>forceRefreshTable()</code> .
<code>lib/db/prefetch.ts</code>	Warms the React Query cache (dashboard, current budget, net worth, debt) from IDB using <code>fetchQuery</code> .
<code>lib/db/*.test.ts</code>	Tests for delta pulls, the generation guard and delete reconciliation.

lib/sync/ — mutation queue

Offline write path.

<code>lib/sync/SyncEngine.ts</code>	Drains <code>sync_queue</code> . Dispatch table (table, op) to server action. Bulk dispatch for SMS inserts. Superseded-update collapse, temp-id dependency gating, per-record serialization, bounded concurrency (4), retry with exponential backoff (3 tries), rollback, temp-to-real id swap with child foreign-key rewrite.
<code>lib/sync/reconcile.ts</code>	Builds the IDB row when an INSERT syncs, so local state the user advanced before the sync (e.g. an SMS categorized while its INSERT was queued) survives.
<code>lib/sync/deviceFields.ts</code>	Device-only columns (<code>sms_transactions.raw_text</code> and <code>sender</code>) that must survive a server row overwrite.
<code>lib/sync/*.test.ts</code>	Queue-drain ordering, batching, collapse and reconcile tests.

lib/providers/ — React context

App-wide providers.

<code>lib/providers/SyncProvider.tsx</code>	Owns the SyncEngine. Fast path (render from cache, reconcile in background) vs cold path (hydrate first). Foreground/resume refresh, coalesced post-sync refresh, onSynced/onRollback invalidation map, daily touchLastSeen().
<code>lib/providers/QueryProvider.tsx</code>	QueryClient: staleTime Infinity, gcTime 10 minutes, no refetch on focus, retry 1. IDB is the source of truth.
<code>lib/providers/CurrencyProvider.tsx</code>	Current user's currency from the profile.
<code>lib/providers/AccentProvider.tsx</code>	Accent preset state, persisted to localStorage (allocat-accent).
<code>lib/providers/ThemeProvider.tsx</code>	next-themes wrapper (light/dark).
<code>lib/providers/QuickActionProvider.tsx</code>	Lets a route register the dock's quick-action button.

lib/hooks/ — read and mutation hooks

Each domain has read hooks (IDB first, then server action) and mutation hooks (optimistic IDB write, enqueue, invalidate).

<code>lib/hooks/useBudget.ts</code>	<code>budgetKey(month, year)</code> , <code>getBudgetFromIDB</code> , <code>useBudgetData</code> , templates, and every budget mutation (<code>setup</code> , <code>total</code> , <code>category add/allocation/icon/color/name/delete</code> , <code>item add/update/delete</code>).
<code>lib/hooks/useBudgetCarry.ts</code>	Carry-forward marker, source lookup, carry, undo and dismiss.
<code>lib/hooks/useCategoryData.ts</code>	Category detail read that resolves a <code>temp_id</code> through <code>id_map</code> .
<code>lib/hooks/useDashboard.ts</code>	<code>DASHBOARD_KEY</code> , <code>getDashboardFromIDB</code> , <code>useDashboardData</code> , <code>useQuickLogSpend (PAYMENT)</code> , category items.
<code>lib/hooks/useDebt.ts</code>	<code>DEBT_KEY</code> , debt reads, <code>add/update/delete</code> , <code>makePayment</code> , payment trend.
<code>lib/hooks/useGoals.ts</code>	Goals as assets with <code>is_goal</code> : CRUD and achieve.
<code>lib/hooks/useNetWorth.ts / useAssetCategories.ts / useAssetHistory.ts</code>	Net worth, asset categories and asset value history reads and mutations.
<code>lib/hooks/useSmsTransactions.ts</code>	SMS query keys, <code>pending/categorized/all/item</code> reads from IDB, <code>categorize/ignore/delete/recategorize/report-mistake</code> mutations, <code>invalidateSmsCaches</code> .
<code>lib/hooks/useActivityLogs.ts</code>	Activity logs, per-item activity, debt payments, month grouping.
<code>lib/hooks/useReports.ts</code>	Monthly report read and notes save.

<code>lib/hooks/useProfile.ts / useUpdateAvatar.ts / useUpdateCurrency.ts</code>	Profile read and optimistic profile edits.
<code>lib/hooks/useNotifications.ts</code>	Local inbox reads, unread count, mark read.
<code>lib/hooks/useSync.ts</code>	<code>useSync()</code> (pending count, online, hydrated) and <code>useEnqueue()</code> .
<code>lib/hooks/useAppFlags.ts</code>	Runtime flags from <code>/api/app-config</code> (fails open to defaults, refetched hourly).
<code>lib/hooks/useSupporter.ts</code>	<code>useIsSupporter</code> from the profile row (cosmetic only).
<code>lib/hooks/useFormatCurrency.ts</code>	Formatter bound to the user's currency.
<code>lib/hooks/useHaptic.ts / useIsNative.ts / usePWAIInstall.ts</code>	Haptics, an SSR-safe native flag, and the PWA install prompt.

lib/actions/ — server actions

"use server" modules. The only client-reachable code that talks to Supabase.

<code>lib/actions/budget.ts</code>	Budgets, categories and items: <code>get/view/ensure</code> row, CRUD, allocation validation, <code>quickLogSpend</code> (with the asset/debt cascade, activity log and overspend notify), <code>reverseSpend</code> , <code>setupBudgetFromTemplate</code> , <code>carryBudgetForward</code> , <code>undoCarriedBudget</code> .
<code>lib/actions/budget-templates.ts</code>	Custom templates CRUD plus stamping durable template identity onto budgets and items.
<code>lib/actions/sms.ts</code>	SMS ingest (single and bulk), categorize, recategorize, unallocate, ignore, delete, report mistake (blocklist), near-limit notify.
<code>lib/actions/debt.ts</code>	Debts CRUD, <code>makePayment</code> , <code>reverseDebtPayment</code> , payment trend.
<code>lib/actions/net-worth.ts</code>	Assets and goals CRUD, <code>achieveGoalAsset</code> , net worth data.
<code>lib/actions/asset-categories.ts / asset-history.ts</code>	Asset categories CRUD and seeding. Asset entries and the daily snapshot upsert.
<code>lib/actions/dashboard.ts</code>	Dashboard aggregate read (server fallback).
<code>lib/actions/profile.ts</code>	Onboarded flag, currency, avatar, <code>touchLastSeen</code> (<code>last_seen_at</code> , <code>last_app_mode</code> , <code>user_active_days</code>).
<code>lib/actions/auth.ts</code>	login, signup, <code>signOut</code> , <code>deleteAccount</code> .
<code>lib/actions/push.ts</code>	Web Push subscribe/unsubscribe/test and FCM token register/unregister.
<code>lib/actions/ai-chat.ts</code>	<code>buildFinancialContext</code> : topic-scoped slices of budget, goals, net worth, debts and reports for the chat prompt.
<code>lib/actions/insights.ts / monthly-summary.ts / notify-messages.ts</code>	<code>OpenRouter</code> wrappers for the weekly insight, the monthly report summary and the overspend message. All return null on failure (static fallback).
<code>lib/actions/reports.ts / activity-logs.ts / feedback.ts / support.ts</code>	Report upsert, activity reads, feedback insert, supporter reconcile.

lib/supabase/ — clients and auth

Supabase clients per environment.

<code>lib/supabase/server.ts</code>	<code>createClient()</code> and <code>getAuthedUser()</code> , both wrapped in <code>React cache()</code> so auth is verified once per request.
<code>lib/supabase/client.ts</code>	Browser client (<code>createBrowserClient</code>).

<code>lib/supabase/service.ts</code>	Service-role client that bypasses RLS. Server-only, for cross-user work.
<code>lib/supabase/middleware.ts</code>	<code>updateSession()</code> : refreshes the session cookie and redirects protected and auth routes. <code>skipAuth()</code> for server actions and <code>/api</code> .

lib/sms/ — SMS pipeline (web side)

Pure logic plus the client ingest.

<code>lib/sms/ingestClient.ts</code>	Client ingest: OTP drop, dedupe and template keys, blocklist, in-flight guard, regex parse, debit-only, rule match plus period resolution, atomic IDB insert with optimistic spend, enqueue (no raw text), local notifications. Also <code>reapplyRulesToPending</code> .
<code>lib/sms/match.ts</code>	<code>normalizeMerchant</code> , <code>matchMerchantRules</code> (exact > contains > regex), <code>txnDedupeKey</code> , <code>smsTemplateKey</code> (hash53 of sender plus masked skeleton).
<code>lib/sms/resolveRuleItem.ts</code>	Resolves a rule to THIS month's budget item through durable template identity (items get new UUIDs each month).
<code>lib/sms/nativeMirror.ts</code>	Builds and pushes the native closed-app mirror: rules resolved to current-month items, quick targets, config.
<code>lib/sms/monthAllocations.ts</code>	Groups categorized transactions by budget item for a month.
<code>lib/sms/nearLimit.ts</code>	Near-limit (at least 90%) and pace-projection checks from IDB.
<code>lib/sms/amountDelta.ts</code>	Math for editing a transaction amount (keeps <code>actual_amount</code> and <code>original_amount</code> consistent).
<code>lib/sms/insightStats.ts</code> / <code>recap.ts</code>	Weekly stats from IDB, and scheduling the Sunday 19:00 weekly insight local notification.
<code>lib/sms/notifPrefs.ts</code>	Device-local notification preferences.
<code>lib/sms/bulkIngest.ts</code>	<code>MAX_BULK_INGEST</code> = 50 and the <code>BulkIngestOutcome</code> type.

lib/ai/ — AI helpers

Prompts, guards and parsers (pure, unit-tested).

<code>lib/ai/guard.ts</code>	Chat security: <code>sanitizeMessages</code> , <code>isPromptExtraction</code> , <code>isOffTopic</code> , <code>wrapUntrustedData</code> , and a leak-guard stream that cuts output if the system prompt leaks.
<code>lib/ai/parseSmsTransaction.ts</code>	Regex SMS parser: amount, currency, merchant, direction, date, confidence. OTP detection. Mirrored in <code>SmsParser.java</code> .
<code>lib/ai/parseSpend.ts</code>	Regex parser for shared text (share target).
<code>lib/ai/insightPrompt.ts</code> / <code>monthlySummaryPrompt.ts</code> / <code>overspendPrompt.ts</code>	Prompt builders and response parsers for the three AI notifications.

lib/budget/ — budget domain logic

Pure, tested functions.

<code>lib/budget/setupMath.ts</code>	Percent to amount allocation, template to setup categories, rebalance, fit items.
<code>lib/budget/carry.ts</code>	Carry-forward: find the source month, build the <code>CARRY_SETUP</code> payload with spend reset.

<code>lib/budget/diffTemplateToBudget.ts / templateDrift.ts</code>	Template to budget diff (apply edits) and structural drift detection.
<code>lib/budget/quizMapping.ts / quizDraft.ts</code>	Quiz answers to a 3-category plan, and the local draft store.
<code>lib/budget/categorySuggestions.ts / saveTemplateDurableId.ts</code>	Item name suggestions and durable id selection when saving a template.
<code>lib/budget-templates.ts</code>	Predefined templates with durable item ids.

lib/ — other modules

Everything else under lib/.

<code>lib/server/activity-logger.ts</code>	<code>logActivity()</code> writes <code>activity_logs</code> . <code>fmt()</code> and <code>getUserCurrency()</code> for money strings.
<code>lib/server/push-notify.ts</code>	<code>notifyUser()</code> : single-user Web Push (VAPID). Silent on failure. No FCM (would double-notify).
<code>lib/server/push-broadcast.ts</code>	Admin fan-out over Web Push and FCM with segments (all, android, web, supporters, inactive_7d), chunked.
<code>lib/server/fcm.ts</code>	FCM HTTP v1 sender. Self-signed JWT exchanged for an OAuth token. Prunes tokens on 404/UNREGISTERED only.
<code>lib/server/openrouter.ts</code>	Single OpenRouter call site (model <code>openrouter/free</code>).
<code>lib/server/rateLimit.ts</code>	In-memory sliding-window limiter (per instance).
<code>lib/admin/guard.ts / queries.ts / actions.ts / format.ts</code>	Admin allowlist gate (404), service-role reads through <code>admin_*</code> RPCs, guarded writes, formatting helpers.
<code>lib/config/flags.ts / serverFlags.ts</code>	Flag types, defaults and tolerant parse. Server read memoized 60s, fails open.
<code>lib/native/SmsReader.ts</code>	TypeScript interface for the native <code>SmsReader</code> plugin.
<code>lib/native/notify.ts / notifSounds.ts</code>	<code>notifyLocal</code> (native local notification, no-op on web) and the sound catalog and channels.
<code>lib/native/debtReminders.ts / sheetRegistry.ts</code>	Scheduled lent-money reminders, and the back-button stack for custom sheets.
<code>lib/notify/history.ts / messages.ts</code>	Local notification inbox (50 rows, 30-day TTL) with <code>emitNotification</code> , and cat-voice message pools with overspend tiers.
<code>lib/play/installs.ts / csv.ts</code>	Play install stats from GCS (UTF-16LE CSV parse).
<code>lib/tour/*</code>	<code>driver.js</code> tour: context (localStorage <code>allocat-tour-state</code>), steps, mock data, hooks.
<code>lib/theme/accents.ts / dataViz.ts</code>	Accent presets and the categorical chart palette.
<code>lib/currency/catalog.ts / lib/number-format.ts</code>	Currency definitions and Intl-based money formatting.
<code>lib/auth/clearSession.ts</code>	Logout cleanup: flush the queue, clear IDB and account-scoped localStorage keys.
<code>lib/support/links.ts / local.ts</code>	Ko-fi URLs, and patching the supporter flag into the IDB profile.
<code>lib/utils/*</code>	<code>budget-cascade</code> (IDB mirror of the linked spend cascade), <code>budget-completion</code> , <code>dashboard-empty</code> , <code>debt-calc</code> (EMI), <code>greeting</code> , <code>link-suggest</code> , <code>netWorthHistory</code> , <code>urlBase64</code> (VAPID), <code>uuid</code> (randomUUID polyfill).
<code>lib/nav/navItems.ts / lib/profile/avatars.ts</code>	The 5 tabs in order, and the avatar catalog.

<code>lib/types/database.ts</code>	Generated Supabase Database type (all tables and RPCs).
<code>lib/ai-utils.ts</code>	<code>detectTopic()</code> : picks which financial slices the chat needs.

android/ — native Capacitor shell

Java code under `android/app/src/main/java/com/octane/allocat/`.

<code>MainActivity.java</code>	Capacitor BridgeActivity. Registers <code>SmsReaderPlugin</code> before the bridge starts and tracks <code>WebView</code> liveness.
<code>SmsTransactionReceiver.java</code>	<code>BroadcastReceiver</code> for <code>SMS_RECEIVED</code> (also when the app is killed). Assemble PDUs, <code>SmsFilter</code> , parse, debit-only, blocklist, queue. If the <code>WebView</code> is alive, emit to JS; if not, match rules and post a native notification.
<code>SmsFilter.java</code>	On-device compliance gate: only financial-looking, non-OTP SMS pass.
<code>SmsParser.java</code>	Java port of <code>parseSmsTransaction.ts</code> (amount, merchant, direction).
<code>SmsQueue.java</code>	<code>SharedPreferences</code> queue of captured SMS, drained by JS on open.
<code>SmsReaderPlugin.java</code>	Capacitor plugin: permissions, <code>getQueued</code> , <code>setRules</code> , <code>setConfig</code> , <code>setBlocklist</code> , <code>setQuickTargets</code> , and the <code>smsReceived</code> event.
<code>SmsRules.java</code> / <code>SmsTargets.java</code> / <code>SmsConfig.java</code> / <code>SmsBlocklist.java</code>	Native mirrors pushed from JS: rules plus budget state, quick-allocate targets, notification config, blocked template keys.
<code>SmsHash.java</code> / <code>SmsSignature.java</code>	Byte-compatible ports of <code>txnDedupeKey</code> and <code>smsTemplateKey</code> (hash53).
<code>SmsNotifier.java</code> / <code>SmsMessages.java</code>	Posts heads-up notifications. Cat-voice message pools mirror <code>lib/notify/messages.ts</code> .
<code>SmsAccum.java</code> / <code>SmsOverspend.java</code>	Running totals of spend and overspend across closed-app transactions until the next app open.
<code>AndroidManifest.xml</code>	Permissions (<code>RECEIVE_SMS</code> only, no <code>READ_SMS</code>), the receiver, deep-link scheme, FCM default channel.
<code>build.gradle</code> files, <code>google-services.json</code> , <code>res/</code>	Gradle build, Firebase config (not secret), icons, splash, notification sounds (<code>res/raw</code>).

supabase/migrations/ — database history

SQL migrations in time order.

<code>20260101000000_init.sql</code>	Initial schema: profiles, budgets, categories, <code>budget_items</code> , <code>asset_categories</code> , <code>assets</code> , <code>asset_value_history</code> , <code>debts</code> , <code>reports</code> , <code>net_worth_snapshots</code> , <code>activity_logs</code> , <code>push_subscriptions</code> . Indexes, RLS, the <code>updated_at</code> trigger, and the <code>handle_new_user</code> profile trigger.
<code>20260603000000_sms_transactions_merchant_rules.sql</code>	<code>sms_transactions</code> and <code>merchant_rules</code> .
<code>20260605000000_sms_raw_text_nullable.sql</code>	<code>raw_text</code> nullable, and purges stored raw SMS (privacy).
<code>20260607 / 0609 / 0615</code>	Entity color key, budget item emoji, SMS transaction label.
<code>20260608000000_budget_templates.sql</code>	User budget templates table (JSONB categories).
<code>20260620000000_sms_blocklist.sql</code>	Hashed template blocklist.
<code>20260621000000_transactions_unify_feedback.sql</code>	source and <code>original_amount</code> on transactions, the feedback table, a debt due-date index.

20260622000000_durable_merchant_rules.sql	Durable (template_id, template_item_id) identity so rules survive months.
20260630*/*.sql	app_source column (later dropped) and app_config for force update.
20260701 / 0723	overspend_count on items, profiles.app_mode.
20260908000000_admin_portal.sql	Admin tables (landing_events, play_install_stats, push_campaigns) and security-definer admin_* functions.
20260908010000_user_active_days.sql	Per-day active history for DAU/WAU/MAU.
20260908020000_fcm_tokens.sql	FCM tokens table and admin_push_reach.
20260911000000_admin_user_search_paged.sql	Paged and sortable admin user search.
20260912000000_delta_sync.sql	updated_at plus a BEFORE UPDATE trigger on every synced table (delta watermark source).
20260913000000_sms_template_key.sql	Persists template_key on the transaction row.
docs/migrations/*.sql	Older out-of-band migrations: profile currency, the removed subscription, supporters.

docs/ and public/

Documentation and static assets.

docs/sms-feature.md	Full SMS feature design.
docs/product-overview.md	Product overview.
docs/play-*.md / console-setup.md / play-store-compliance-report.md	Play release process, console setup, SMS policy compliance.
docs/superpowers/specs/*	Design specs: multi-currency, delta sync, reliability and scale plan.
docs/release-notes/*	Release notes (subscription, then free forever).
public/	Icons, logo, self-hosted fonts, Lottie animations, profile avatars, notification sounds, generated sw.js.

28 Glossary

Term	Meaning
IDB	IndexedDB, the browser database. Accessed through Dexie.
Hydrate	Pull server rows into IDB (<code>hydrateAllTables</code>).
Watermark	Newest server <code>updated_at</code> a table has pulled; the next pull asks only for rows at or after it.
Id sweep	An id-only query used to detect rows deleted on the server.
Optimistic write	Updating IDB before the server confirms.
Temp id	<code>temp_<uuid></code> given to a locally created row until its INSERT syncs.
id_map	IDB table mapping temp ids to real server ids.
Protected id	A row with an un-synced local write; hydration won't overwrite or delete it.
Superseded write	An older queued UPDATE made redundant by the next one; dropped unsent.
Drain	One run of <code>processQueue()</code> until no item is ready.
Rollback	Undoing an optimistic write after 3 failed attempts.
Wipe generation	Counter bumped by <code>clearDB()</code> so in-flight pulls discard their data.
Cascade	A server-side write that changes other tables (spend → asset/debt/activity).
Dedupe key	Hash of sender + SMS body. Same SMS, same key.
Template key	Hash of sender + masked SMS skeleton. Same kind of SMS, same key (used for the blocklist).
Merchant rule	Learned mapping from a merchant pattern to a budget item (exact, contains or regex).
Durable identity	<code>(template_id, template_item_id)</code> : stays the same across months while item ids change.
Native mirror	Snapshot of rules, targets, config and blocklist pushed from JS to Java for closed-app notifications.
Remote-URL mode	The Capacitor shell loads the live website instead of bundled files.
Runtime flag	Kill switch stored in the database, effective without a redeploy.
RLS	Postgres row-level security; here "each user sees only their rows".
Carry-forward	Copying last month's budget structure into an empty month.

End of handbook.